

本地部署开源大模型

Ch.5 在Ubuntu 22.04系统下ChatGLM3-6B高效微调实战

无论是在单机单卡（一台机器上只有一块GPU）还是单机多卡（一台机器上有多块GPU）的硬件配置上启动ChatGLM3-6B模型，其前置环境配置和项目文件是相同的。如果大家对配置过程还不熟悉，建议参考我在上一期直播《在Ubuntu 22.04系统下部署运行ChatGLM3-6B模型》中的部署视频和详细课件。请按照以下步骤进行配置和验证，以确保顺利启动模型：

1. 执行Ubuntu初始化配置：更改国内软件源 --> 软件包更新 --> 设置英文目录 --> 安装Chrome浏览器（非必要，但建议） --> 配置VPN；
2. 配置大模型运行环境：安装显卡驱动 --> 安装Anaconda环境；

完成初始配置后，大家需要根据自己实际使用的硬件环境，选择相应的部署和运行步骤：

• 单机单卡情况

参考《在Ubuntu 22.04系统下部署运行ChatGLM3-6B模型》课件，重点关注 [四、ChatGLM3-6B私有化部署](#) 和 [五、运行ChatGLM3-6B模型的方式](#) 章节。

• 单机多卡情况

先遵循本课件中为单机多卡情况提供的部署指南，执行多卡环境下ChatGLM3-6B模型的启动步骤。

1. 本地化部署ChatGLM3-6B模型

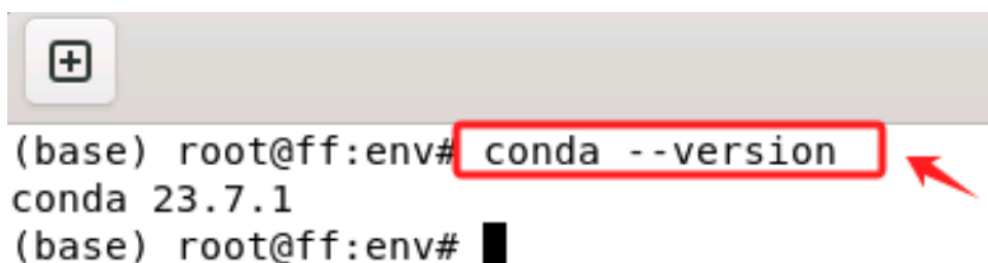
如果跟着上一期公开课及课件实践过单机单卡的操作流程，我们建议针对本期内容的单机多卡也设置**创建一个新的虚拟环境**。这样做可以有效避免版本冲突和依赖问题，确保多卡环境具备专门优化的、独立于单卡环境的配置，简化项目的维护和调试过程。

在实际的生产开发中，一个独立项目对应一个单独的隔离环境是一个比较标准的做法，也建议大家以后在做项目开发的时候遵循。具体的部署流程如下：

• Step 1. 更新Conda

首先，打开命令行终端，检查 Conda 的版本，输入如下命令：

```
conda --version
```



这条命令都会显示当前安装的 Conda 版本号。如果 Conda 已正确安装并正确设置在环境变量中，会正常输出conda的版本，如果收到类似于“命令未找到”的错误，则说明 Conda 没有被添加到环境变量中，或者根本没有安装 Conda。在这种情况下，请查看《在Ubuntu 22.04系统下部署运行ChatGLM3-6B模型》中 [2.3 安装Anaconda环境](#) 部分的说明。

- Step 2. 升级Conda到最新版本

在更新过程中，系统会询问是否要继续进行，需要输入 `y` 来确认。使用 Conda 自身的更新命令执行更新：

```
conda update conda
```

```
(base) root@ff:env# conda update conda
Collecting package metadata (current_repodata.json): done
Solving environment: done

==> WARNING: A newer version of conda exists. <==
  current version: 23.7.1 → 当前版本
  latest version: 23.11.0 → 最新版本
```

- Step 3. 检查Conda更新情况

更新完成后，再次检查 Conda 的版本来确认更新是否成功。

```
conda --version
```

```
(base) root@ff:env#
(base) root@ff:env# conda --version
conda 23.11.0
(base) root@ff:env#
```

- Step 4. 使用Conda更新软件包

更新完 Conda 后，需要更新环境中的所有包，以确保所有软件包都是最新的。避免产生未知的依赖问题，使用以下命令来更新所有安装的包：

```
conda update --all
```

```
(base) root@ff:env# conda update --all
Channels:
  - defaults
  - https://mirrors.tuna.tsinghua.edu.cn/anaconda/pkgs/free
  - pytorch
  - conda-forge
Platform: linux-64
Collecting package metadata (repodata.json): done
Solving environment: done
```

- Step 5. 使用Conda创建独立的隔离环境

创建一个新环境用于多卡部署启动ChatGLM3-6B，避免与现有环境中的包发生冲突。使用以下命令创建一个新环境（我这里设置的环境名为 `chatglm3_multi`，大家根据需要更改虚拟环境的名称）：

```
conda create --name chatglm3_multi python=3.11
```

```
(base) root@ff:env# conda create -n chatglm3_multi python=3.11
Channels:
- defaults
- https://mirrors.tuna.tsinghua.edu.cn/anaconda/pkgs/free
- pytorch
- conda-forge
Platform: linux-64
Collecting package metadata (repodata.json): done
Solving environment: /
```

- Step 6. 进入隔离环境

创建完成后，使用 `conda activate` 进入该虚拟环境。

```
# To activate this environment, use
#
# $ conda activate chatglm3_multi
#
# To deactivate an active environment, use
#
# $ conda deactivate
```

```
(base) root@ff:env# ls
(base) root@ff:env# conda activate chatglm3_multi
(chatglm3_multi) root@ff:env#
```

除此之外，大家一定要注意：如果使用远程连接，关闭了当前终端，或者是重启了电脑等情况后，再次启动ChatGLM3-6B模型服务时，需要先进入这个虚拟环境。进入指定的虚拟环境方法如下：

```
+ 这是当前的默认虚拟环境
root@ff: ~
(base) root@ff:~# conda env list
# conda environments:
#
base                * /home/util/anaconda3
BERT                /home/util/anaconda3/envs/BERT
Dinetpre            /home/util/anaconda3/envs/Dinetpre
Fay                 /home/util/anaconda3/envs/Fay
LangChain           /home/util/anaconda3/envs/LangChain
bark                /home/util/anaconda3/envs/bark
base_clone          /home/util/anaconda3/envs/base_clone
chat6B              /home/util/anaconda3/envs/chat6B
chatglm2            /home/util/anaconda3/envs/chatglm2
chatglm3            /home/util/anaconda3/envs/chatglm3
chatglm3_multi      /home/util/anaconda3/envs/chatglm3_multi
chatglm6bQY         /home/util/anaconda3/envs/chatglm6bQY
```

使用 `conda activate + 指定虚拟环境名称` 的方式，进入该虚拟环境。如果命令行最前面已显示该虚拟环境，说明进入成功。

```
(base) root@ff:~#
(base) root@ff:~# conda activate chatglm3_multi
(chatglm3_multi) root@ff:~#
(chatglm3_multi) root@ff:~#
```

- Step 7. 在虚拟环境中安装Pytorch

在上一期视频中说过，安装GPU版本的Pytorch需要根据当前安装的显卡驱动最高可支持的CUDA版本来选择正确的Pytorch版本，所以先通过 `nvidia-smi` 命令查看一下：

```
(chatglm3_multi) root@ff:env# nvidia-smi
```

Mon Jan 8 11:42:58 2024

NVIDIA-SMI 525.116.03		Driver Version: 525.116.03		CUDA Version: 12.0	
GPU	Name	Persistence-M	Bus-Id	Disp.A	Volatile Uncorr. ECC
Fan	Temp	Perf	Pwr:Usage/Cap	Memory-Usage	GPU-Util Compute M.
					MIG M.
0	NVIDIA GeForce ...	Off	00000000:0A:00.0	Off	N/A
62%	60C	P2	294W / 350W	4823MiB / 24576MiB	80% Default
					N/A

在我的这台机器上，最高可支持的CUDA版本是12.0，需要根据此限制，进入Pytorch官网：<https://pytorch.org/get-started/previous-versions/> 选择合适的Pytorch版本。注：这里大家要根据自己的实际情况灵活的选择适合自己的Pytorch版本。

Linux and Windows

```
# CUDA 11.8
conda install pytorch==2.1.1 torchvision==0.16.1 torchaudio==2.1.1 pytorch-cuda=11.8 -c pytorch -c nvidia
# CUDA 12.1
conda install pytorch==2.1.1 torchvision==0.16.1 torchaudio==2.1.1 pytorch-cuda=12.1 -c pytorch -c nvidia
# CPU Only
conda install pytorch==2.1.1 torchvision==0.16.1 torchaudio==2.1.1 cpuonly -c pytorch
```

直接复制安装命令，进入终端执行。

```
(chatglm3_multi) root@ff:env# conda install pytorch==2.1.1 torchvision==0.16.1 torchaudio==2.1.1 pytorch-cuda=11.8 -c pytorch -c nvidia
Channels:
- pytorch
- nvidia
- defaults
- https://mirrors.tuna.tsinghua.edu.cn/anaconda/pkg/free
- conda-forge
Platform: linux-64
Collecting package metadata (repodata.json): -
```

• Step 8. 检查Pytorch安装是否成功

安装完成后，务必检查是否成功安装了GPU版本的PyTorch。最简单的验证方法如下：

```
import torch
print(torch.cuda.is_available())
```

```
(chatglm3_multi) root@ff:env# python
Python 3.11.7 (main, Dec 15 2023, 18:12:31) [GCC 11.2.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> import torch
>>> print(torch.cuda.is_available())
True
```

输入这两条命令

如果输出是 True，则表示GPU版本的PyTorch已经安装成功并且可以使用CUDA，如果输出是 False，则表明没有安装GPU版本的PyTorch，或者CUDA环境没有正确配置，如果出现这种情况，请重新检查自己的安装过程，并确保此处可以正常加载GPU版本的Pytorch，否则后面的操作会无法执行。

• Step 9. 下载ChatGLM3-6B模型的项目文件

首先，创建一个文件夹来存储该项目文件。

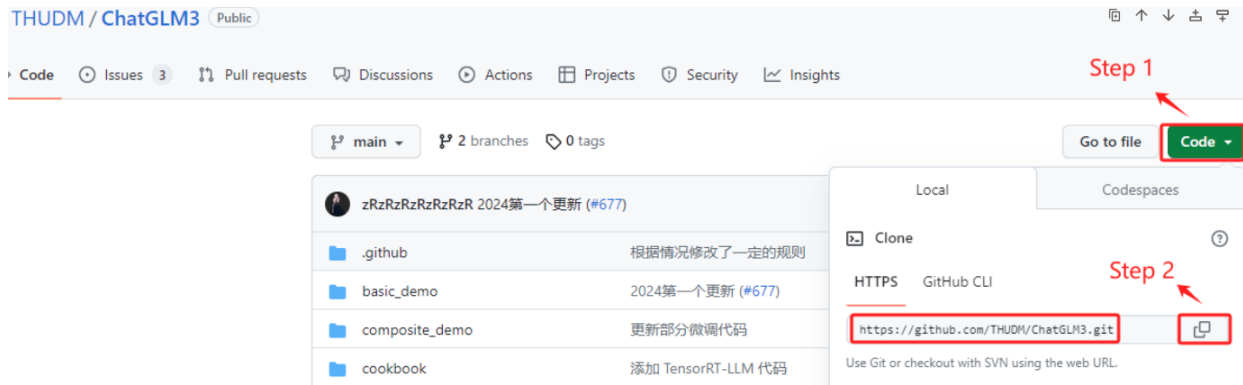
```
(chatglm3_multi) root@ff:00.Work_muyu# mkdir muyu_chatglm3_6b
(chatglm3_multi) root@ff:00.Work_muyu# cd muyu_chatglm3_6b/
(chatglm3_multi) root@ff:muyu_chatglm3_6b#
```

使用git工具在Github拉取ChatGLM3-6B模型的项目文件至本地。如果没有安装git的话，需要先安装git，命令如下：

```
sudo apt install git
```

```
(chatglm3_multi) root@ff:muyu_chatglm3_6b# sudo apt install git
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
git is already the newest version (1:2.34.1-1ubuntu1.10).
git set to manually installed.
The following packages were automatically installed and are no longer required:
  linux-generic-hwe-22.04 linux-headers-5.15.0-23 linux-headers-generic-hwe-22.04 linux-image-generic-hwe-22.04
Use 'sudo apt autoremove' to remove them.
0 upgraded, 0 newly installed, 0 to remove and 5 not upgraded.
(chatglm3_multi) root@ff:muyu_chatglm3_6b#
```

在ChatGLM3-6B的GitHub官网找到远程仓库的url: <https://github.com/THUDM/ChatGLM3>



使用安装好的git工具，将云端的ChatGLM3-6B模型的项目文件拉取到本地环境，执行如下命令：

```
git clone https://github.com/THUDM/ChatGLM3.git
```

```
(chatglm3_multi) root@ff:muyu_chatglm3_6b# git clone https://github.com/THUDM/ChatGLM3.git
Cloning into 'ChatGLM3'...
remote: Enumerating objects: 946, done.
remote: Counting objects: 100% (528/528), done.
remote: Compressing objects: 100% (171/171), done.
Receiving objects: 28% (266/946), 8.71 MiB | 5.35 MiB/s
```

• Step 10. 验证ChatGLM3-6B模型项目文件的完整性

等待下载完成，进入文件夹后验证项目文件的有效性。如执行过程正常的话，在本地会出现 ChatGLM3 文件夹，进入该文件夹，所有的项目文件如下所示：

```
(chatglm3_multi) root@ff:muyu_chatglm3_6b# ll
total 0
drwxr-xr-x 2 root root 0 1月 8 11:51 ./
drwxr-xr-x 2 root root 0 1月 8 11:49 ../
drwxr-xr-x 2 root root 0 1月 8 11:51 ChatGLM3/
(chatglm3_multi) root@ff:muyu_chatglm3_6b# cd ChatGLM3/
(chatglm3_multi) root@ff:ChatGLM3# ll
total 96
drwxr-xr-x 2 root root 0 1月 8 11:51 ./
drwxr-xr-x 2 root root 0 1月 8 11:51 ../
drwxr-xr-x 2 root root 0 1月 8 11:51 basic_demo/
drwxr-xr-x 2 root root 0 1月 8 11:51 composite_demo/
drwxr-xr-x 2 root root 0 1月 8 11:51 cookbook/
-rwxr-xr-x 1 root root 2304 1月 8 11:51 DEPLOYMENT_en.md*
-rwxr-xr-x 1 root root 2098 1月 8 11:51 DEPLOYMENT.md*
drwxr-xr-x 2 root root 0 1月 8 11:51 finetune_basemodel_demo/
drwxr-xr-x 2 root root 0 1月 8 11:51 finetune_chatmodel_demo/
drwxr-xr-x 2 root root 0 1月 8 11:51 .git/
drwxr-xr-x 2 root root 0 1月 8 11:51 .github/
-rwxr-xr-x 1 root root 175 1月 8 11:51 .gitignore*
drwxr-xr-x 2 root root 0 1月 8 11:51 langchain_demo/
-rwxr-xr-x 1 root root 11353 1月 8 11:51 LICENSE*
-rwxr-xr-x 1 root root 4133 1月 8 11:51 MODEL_LICENSE*
drwxr-xr-x 2 root root 0 1月 8 11:51 openai_api_demo/
-rwxr-xr-x 1 root root 7118 1月 8 11:51 PROMPT_en.md*
-rwxr-xr-x 1 root root 6885 1月 8 11:51 PROMPT.md*
-rwxr-xr-x 1 root root 17801 1月 8 11:51 README_en.md*
-rwxr-xr-x 1 root root 17114 1月 8 11:51 README.md*
-rwxr-xr-x 1 root root 474 1月 8 11:51 requirements.txt*
drwxr-xr-x 2 root root 0 1月 8 11:51 resources/
drwxr-xr-x 2 root root 0 1月 8 11:51 tensorrt_llm_demo/
drwxr-xr-x 2 root root 0 1月 8 11:51 tools_using_demo/
-rwxr-xr-x 1 root root 152 1月 8 11:51 update_requirements.sh*
(chatglm3_multi) root@ff:ChatGLM3#
```

进入该文件夹

ChatGLM3 项目文件

• Step 11. 安装ChatGLM3-6B模型项目运行环境的依赖

在项目文件中，有一个 `requirements.txt` 文件，其中包含了该项目所有的依赖项。该文件可以使用 Python 的 `pip` 工具来一键执行安装，因此建议先需要升级 `pip` 包的版本，避免因 `pip` 版本较低导致产生依赖问题。

```
python -m pip install --upgrade pip
```

```
(chatglm3_multi) root@ff:ChatGLM3# python -m pip install --upgrade pip
Requirement already satisfied: pip in /home/util/anaconda3/envs/chatglm3_multi/lib/python3.11/site-packages (23.3.1)
Collecting pip
  Downloading pip-23.3.2-py3-none-any.whl.metadata (3.5 kB)
  Downloading pip-23.3.2-py3-none-any.whl (2.1 MB)
    0.2/2.1 MB 114.0 kB/s eta 0:00:17
```

升级完 `pip` 工具后，执行如下命令一次性安装启动 ChatGLM3-6B 模型的所有依赖包：

```
pip install -r requirements.txt
```

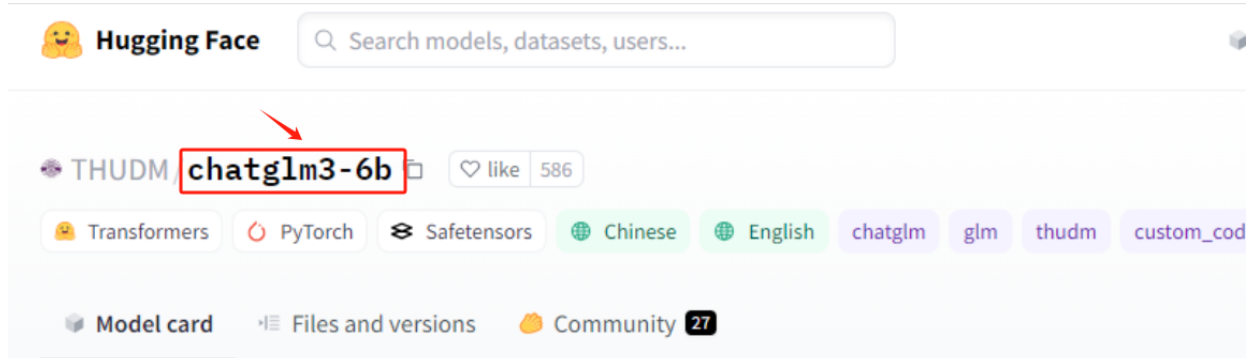
```
(chatglm3_multi) root@ff:ChatGLM3# ls
basic_demo  DEPLOYMENT_en.md  finetune_chatmodel_demo  MODEL_LICENSE  PROMPT.md  requirements.txt  tools_using_demo
composite_demo  DEPLOYMENT.md  langchain_demo  openai_api_demo  README_en.md  resources  update_requirements.sh
cookbook  finetune_basemodel_demo  LICENSE  PROMPT_en.md  README.md  tensorrt_llm_demo
(chatglm3_multi) root@ff:ChatGLM3# pip install -r requirements.txt
Collecting protobuf==4.25.1 (from -r requirements.txt (line 3))
  Downloading protobuf-4.25.1-cp37-abi3-manylinux2014_x86_64.whl.metadata (541 bytes)
```

• Step 12. 下载ChatGLM3-6B模型的权重文件

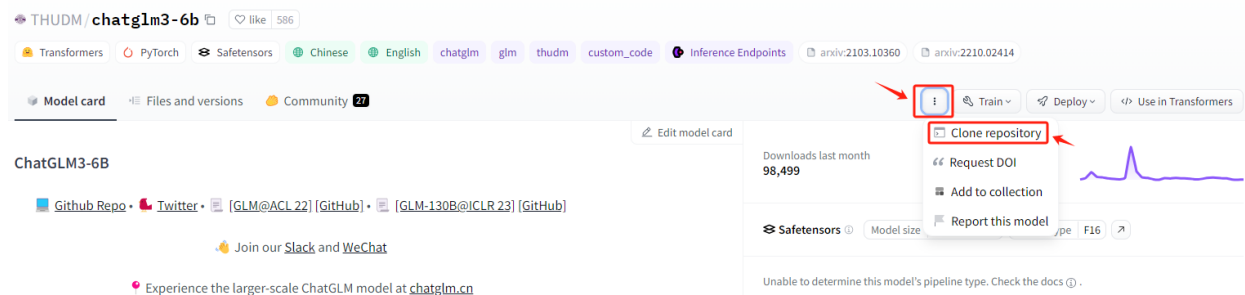
经过 Step 9 的操作过程，我们下载到的只是 ChatGLM3-6B 的一些运行文件和项目代码，并不包含 ChatGLM3-6B 这个模型的权重，还需要进入到 Hugging Face 官网进行下载。下载路径：<https://github.com/THUDM/ChatGLM3>



注：需要开科学上网才能进入Hugging Face官网执行下载，如果没有，可以选择进入 [ModelScope](#) 魔搭社区，按照教程执行下载。



按照如下位置，找到对应的远程仓库的URL。



复制此命令，进入到服务器的命令行准备执行。



如果没有安装过git-lfs这个工具，需要先进行安装，安装命令如下：

```
sudo apt-get install git-lfs
```

```
(chatglm3_multi) root@ff:~# sudo apt-get install git-lfs
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
git-lfs is already the newest version (3.0.2-1ubuntu0.2).
The following packages were automatically installed and are no longer required:
  linux-generic-hwe-22.04 linux-headers-5.15.0-23 linux-headers-5.15.0-23-generic linux-headers-generic-hwe-22.04
Use 'sudo apt autoremove' to remove them.
0 upgraded, 0 newly installed, 0 to remove and 5 not upgraded.
```

初始化git lfs，这是使用git拉取模型权重必要的操作，初始化命令如下：

```
git lfs install
```

```
(chatglm3_multi) root@ff:~# sudo git lfs install
Git LFS initialized.
(chatglm3_multi) root@ff:~#
```

完成git-lfs的初始化后，直接复制 Hugging Face上提供的命令，在终端运行，等待下载完成即可。

```
git clone https://huggingface.co/THUDM/chatglm3-6b
```

```
(chatglm3_multi) root@ff:~# sudo git clone git clone https://huggingface.co/THUDM/chatglm3-6b
```

等待下载完成后，在 ChatGLM3 目录下出现一个新的 chatglm3-6b 文件夹，里面存放的就是ChatGLM3-6B模型的权重文件。

```
(chatglm3_multi) root@ff:muyu_chatglm3_6b# cd ChatGLM3/
(chatglm3_multi) root@ff:ChatGLM3# ll
total 96
drwxr-xr-x 2 root root    0 1月 8 14:17 ./
drwxr-xr-x 2 root root    0 1月 8 14:17 ../
drwxr-xr-x 2 root root    0 1月 8 11:51 basic_demo/
drwxr-xr-x 2 root root    0 1月 8 14:11 chatglm3-6b/
drwxr-xr-x 2 root root    0 1月 8 11:51 composite_demo/
drwxr-xr-x 2 root root    0 1月 8 11:51 cookbook/
-rwxr-xr-x 1 root root 2304 1月 8 11:51 DEPLOYMENT_en.md*
-rwxr-xr-x 1 root root 2098 1月 8 11:51 DEPLOYMENT.md*
drwxr-xr-x 2 root root    0 1月 8 11:51 finetune_basemodel_demo/
```

模型文件

全部文件如下所示：


```
(chatglm3_multi) root@ff:ChatGLM3# cd chatglm3-6b/
(chatglm3_multi) root@ff:chatglm3-6b# ll
total 24390292
drwxr-xr-x 2 root root      0 1月 10 13:36 ./
drwxr-xr-x 2 root root      0 1月 10 12:22 ../
-rwxr-xr-x 1 root root    1317 1月 8 12:42 config.json*
-rwxr-xr-x 1 root root   2332 1月 8 12:42 configuration_chatglm.py*
-rwxr-xr-x 1 root root   1519 1月 8 12:42 gitattributes*
-rwxr-xr-x 1 root root 1827774160 1月 8 13:31 model-00001-of-00007.safetensors*
-rwxr-xr-x 1 root root 1968291888 1月 8 14:12 model-00002-of-00007.safetensors*
-rwxr-xr-x 1 root root 1927406936 1月 8 14:06 model-00003-of-00007.safetensors*
-rwxr-xr-x 1 root root 1815217640 1月 8 14:28 model-00004-of-00007.safetensors*
-rwxr-xr-x 1 root root 1968291920 1月 8 14:08 model-00005-of-00007.safetensors*
-rwxr-xr-x 1 root root 1927406960 1月 8 14:04 model-00006-of-00007.safetensors*
-rwxr-xr-x 1 root root 1052805400 1月 8 13:06 model-00007-of-00007.safetensors*
-rwxr-xr-x 1 root root   55678 1月 8 12:43 modeling_chatglm.py*
-rwxr-xr-x 1 root root   4133 1月 8 12:42 MODEL_LICENSE*
-rwxr-xr-x 1 root root   21249 1月 8 12:43 model.safetensors.index.json*
-rwxr-xr-x 1 root root 1827781090 1月 8 14:01 pytorch_model-00001-of-00007.bin*
-rwxr-xr-x 1 root root 1968299480 1月 8 14:04 pytorch_model-00002-of-00007.bin*
-rwxr-xr-x 1 root root 1927415036 1月 10 13:36 pytorch_model-00003-of-00007.bin*
-rwxr-xr-x 1 root root 1815225998 1月 8 13:59 pytorch_model-00004-of-00007.bin*
-rwxr-xr-x 1 root root 1968299544 1月 8 14:02 pytorch_model-00005-of-00007.bin*
-rwxr-xr-x 1 root root 1927415036 1月 8 14:09 pytorch_model-00006-of-00007.bin*
-rwxr-xr-x 1 root root 1052808542 1月 8 13:10 pytorch_model-00007-of-00007.bin*
-rwxr-xr-x 1 root root   20437 1月 8 12:45 pytorch_model.bin.index.json*
-rwxr-xr-x 1 root root   14692 1月 8 12:45 quantization.py*
-rwxr-xr-x 1 root root   7376 1月 8 12:42 README.md*
-rwxr-xr-x 1 root root   12977 1月 8 12:45 tokenization_chatglm.py*
-rwxr-xr-x 1 root root     518 1月 8 12:46 tokenizer_config.json*
-rwxr-xr-x 1 root root 1018370 1月 8 12:49 tokenizer.model*
```

如果直接使用git lfs下载的速度过慢，建议直接下载权重文件至本地。一种最简单的方式就是这类大的文件，直接通过浏览器下载到本地后，然后再移动到chatglm3-6b这个文件夹中。这种方式最简单粗暴，且效率也很高。

 pytorch_model-00001-of-00007.bin		1.83 GB			Upload pytorch_model-00001-of-00007.bin
 pytorch_model-00002-of-00007.bin		1.97 GB			Upload pytorch_model-00002-of-00007.bin
 pytorch_model-00003-of-00007.bin		1.93 GB			Upload pytorch_model-00003-of-00007.bin
 pytorch_model-00004-of-00007.bin		1.82 GB			Upload pytorch_model-00004-of-00007.bin
 pytorch_model-00005-of-00007.bin		1.97 GB			Upload pytorch_model-00005-of-00007.bin
 pytorch_model-00006-of-00007.bin		1.93 GB			Upload pytorch_model-00006-of-00007.bin
 pytorch_model-00007-of-00007.bin		1.05 GB			Upload pytorch_model-00007-of-00007.bin
 pytorch_model.bin.index.json		20.4 kB			Add missing file

2. 单机多卡启动ChatGLM3-6B模型

单机多卡（多个 GPU）环境相较于单机单卡（一个 GPU），可以提供更高的计算能力，但同时也会存在更复杂的资源管理和更复杂的程序代码。比如我们需要考虑如何使所有的 GPU 的负载均衡，如果某个 GPU 负载过重，而其他 GPU 空闲，这会导致资源浪费和性能瓶颈，除此之外，还要考虑每个 GPU 的内存不会被过度使用及模型训练过程中GPU 之间的同步和通信。

尽管如此，单机多卡或者多机多卡往往才是工业界实际使用的方式，单机单卡的瓶颈非常有限，所以这方面的内容还是非常有必要掌握的。而如果初次接触，我们需要做的就是：学会有效的使用简单的GPU监控工具来帮助配置一些重要的超参数，例如批大小（batch size），像出现 GPU 内存溢出（即显存不足）等情况，去考虑减小批大小等等。

2.1 如何查看当前机器的GPU数量

方式一：lspci 命令。这是最常用的方法之一，这个命令会显示与图形相关的设备信息，列出所有 PCI 设备，包括 GPU，其执行命令如下：

```
lspci | grep VGA
```

```
(base) root@ff:env# lspci | grep VGA
0a:00.0 VGA compatible controller: NVIDIA Corporation GA102 [GeForce RTX 3090] (rev a1)
0b:00.0 VGA compatible controller: NVIDIA Corporation GA102 [GeForce RTX 3090] (rev a1)
(base) root@ff:env#
```

方式二：如果系统中安装的是 NVIDIA GPU 和驱动程序，最熟知且最直观的 `nvidia-smi` 命令。

NVIDIA-SMI 525.116.03 Driver Version: 525.116.03 CUDA Version: 12.0									
GPU Name		Persistence-M		Bus-Id		Disp.A		Volatile Uncorr. ECC	
Fan	Temp	Perf	Pwr:Usage/Cap			Memory-Usage		GPU-Util	Compute M.
									MIG M.
0	NVIDIA GeForce ...	Off		00000000:0A:00.0	Off				N/A
30%	23C	P8	26W / 350W			81MiB / 24576MiB		0%	Default
									N/A
1	NVIDIA GeForce ...	Off		00000000:0B:00.0	Off				N/A
0%	29C	P8	28W / 390W			6MiB / 24576MiB		0%	Default
									N/A
Processes:									
GPU	GI	CI	PID	Type	Process name				GPU Memory Usage
	ID	ID							
0	N/A	N/A	1395	G	/usr/lib/xorg/Xorg				49MiB
0	N/A	N/A	2006	G	/usr/lib/xorg/Xorg				29MiB
1	N/A	N/A	1395	G	/usr/lib/xorg/Xorg				4MiB

2.2 如何理解GPU性能参数

参数很多，如何理解各个数值的意义及需要关注哪些信息呢？我们首先来看上半部分的输出：

NVIDIA-SMI 525.116.03 Driver Version: 525.116.03 CUDA Version: 12.0									
GPU Name		Persistence-M		Bus-Id		Disp.A		Volatile Uncorr. ECC	
Fan Temp Perf		Pwr:Usage/Cap		Memory-Usage		GPU-Util		Compute M.	
GPU编号		GPU名称		是否开启持续模式				MIG M.	
0	NVIDIA GeForce ...		Off	00000000:0A:00.0 Off				N/A	
30%	23C	P8	26W / 350W	81MiB / 24576MiB		0%		Default	
风扇	温度	性能状态	当前功率/最大功率	已用显存/最大显存		GPU利用率		N/A	
1	NVIDIA GeForce ...		Off	00000000:0B:00.0 Off				N/A	
0%	29C	P8	28W / 390W	6MiB / 24576MiB		0%		Default	
								N/A	

持续模式：耗能大，但是在新的GPU应用启动时，花费的时间更少，这里显示的是off的状态。

性能状态：从P0到P12，P0表示最大性能，P12表示状态最小性能。

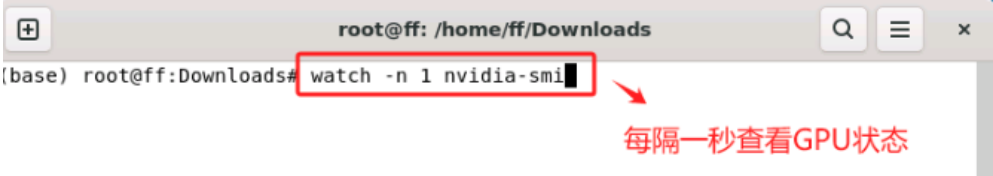
再来看下半部分的输出:

Processes:						
GPU	GI ID	CI ID	PID	Type	Process name	GPU Memory Usage
0	N/A	N/A	1395	G	/usr/lib/xorg/Xorg	49MiB
0	N/A	N/A	2006	G	/usr/lib/xorg/Xorg	29MiB
1	N/A	N/A	1395	G	/usr/lib/xorg/Xorg	4MiB

GPU编号 进程号 进程类型 进程名称 显存占用

除了直接运行 `nvidia-smi` 命令之外，还可以加一些参数，来查看一些本机 GPU 使用的实时情况，这种方式也是在执行训练过程中最简单直观且比较常用的一种监测方式，执行命令如下：

```
watch -n 1 nvidia-smi
```



`-n` 参数可以自己灵活调整，后面添加的数字就是以秒为单位执行一次刷新。

Every 1.0s: nvidia-smi

ff: Tue Jan 9 11:36

→ 每一秒自动刷新一次当前状态

Tue Jan 9 11:30:26 2024

NVIDIA-SMI		525.116.03		Driver Version: 525.116.03		CUDA Version: 12.0			

GPU	Name	Persistence-M	Bus-Id	Disp.A	Volatile	Uncorr.	ECC		
Fan	Temp	Perf	Pwr:Usage/Cap	Memory-Usage	GPU-Util	Compute M.	MIG M.		
=====									
0	NVIDIA GeForce ...	Off	00000000:0A:00.0	Off				N/A	
30%	25C	P8	26W / 350W	6425MiB / 24576MiB	0%	Default	N/A		

1	NVIDIA GeForce ...	Off	00000000:0B:00.0	Off				N/A	
0%	34C	P8	35W / 390W	7116MiB / 24576MiB	0%	Default	N/A		

2.3 单机多卡启动ChatGLM3-6B模型服务

在 Linux 系统中想要在多GPU环境下启动一个应用服务，并且指定使用某些特定的GPU，主要有两种方式：

1. CUDA_VISIBLE_DEVICES环境变量

使用 `CUDA_VISIBLE_DEVICES` 环境变量是最常用的方法之一。这个环境变量可以控制哪些GPU对CUDA程序可见。例如，如果系统有4个GPU（编号为0, 1, 2, 3），而你只想使用编号为1和2的GPU，那么可以在命令行中这样设置：

```
CUDA_VISIBLE_DEVICES=1,2 python your_script.py
```

这会让 `your_script.py` 只看到并使用编号为1和2的GPU。

2. 修改程序代码

这种方式需要直接在代码中设置CUDA设备。例如，在PyTorch中，可以使用 `torch.cuda.set_device()` 函数来指定使用哪个GPU，除此之外，某些框架或工具提供也可能提供相关的参数或环境变量来控制GPU的使用，但都需要修改相关的启动代码。

选择哪种方法取决于具体需求和使用的框架或工具。通常，`CUDA_VISIBLE_DEVICES` 是最简单和最直接的方式，而且它不需要修改代码，这使得它在不同环境和不同应用程序之间非常灵活。如果有控制多个服务并且每个服务需要使用不同GPU的需求，那么需要根据具体情况结合使用。

接下来我们依次尝试上述两种方式启动ChatGLM3-6B模型服务。

- 根据GPU数量自动进行分布式启动

这里我们以命令行的交互方式来进行多卡启动测试。官方提供的脚本名称是：cli_demo.py。

```
(chatglm3_multi) root@ff:basic_demo# ll
total 24
drwxr-xr-x 2 root root 0 1月 8 11:51 ./
drwxr-xr-x 2 root root 0 1月 8 14:17 ../
-rwxr-xr-x 1 root root 3464 1月 8 11:51 cli_batch_request_demo.py*
-rwxr-xr-x 1 root root 3583 1月 8 11:51 cli_demo_bad_word_ids.py*
-rwxr-xr-x 1 root root 1949 1月 8 11:51 cli_demo.py*
-rwxr-xr-x 1 root root 5440 1月 8 11:51 web_demo_gradio.py*
-rwxr-xr-x 1 root root 2957 1月 8 11:51 web_demo_streamlit.py*
(chatglm3_multi) root@ff:basic_demo# pwd
/home/Work/00.Work_muyu/muyu_chatglm3 6b/ChatGLM3/basic_demo
(chatglm3_multi) root@ff:basic_demo# vim cli_demo.py
```

在启动前，仅需要进行一处简单的修改，因为我们已经把ChatGLM3-6B这个模型下载到了本地，所以需要修改一下模型的加载路径。

```
import os
import platform
from transformers import AutoTokenizer, AutoModel
#MODEL_PATH = os.environ.get('MODEL_PATH', 'THUDM/chatglm3-6b')
MODEL_PATH = os.environ.get('MODEL_PATH', './chatglm3-6b')
TOKENIZER_PATH = os.environ.get("TOKENIZER_PATH", MODEL_PATH)

tokenizer = AutoTokenizer.from_pretrained(TOKENIZER_PATH, trust_remote_code=True)
model = AutoModel.from_pretrained(MODEL_PATH, trust_remote_code=True, device_map="auto").eval()

os_name = platform.system()
clear_command = 'cls' if os_name == 'Windows' else 'clear'
stop_stream = False

welcome_prompt = "欢迎使用 ChatGLM3-6B 模型，输入内容即可进行对话，clear 清空对话历史，stop 终止程序"
```

如果仅修改模型权重就执行启动，该过程会自动检测可用的 GPU 并将模型的不同部分映射到这些 GPU 上。状态如下：

```

Yury 1.05: nvidia-smi                                     ff: Tue Jan 9 11:34:10 2024
Tue Jan 9 11:34:10 2024
+-----+
| NVIDIA-SMI 525.116.03   Driver Version: 525.116.03   CUDA Version: 12.0   |
+-----+-----+
| GPU Name      Persistence-M| Bus-Id      Disp.A | Volatile Uncorr. ECC |
| Fan  Temp  Perf  Pwr:Usage/Cap|           Memory-Usage | GPU-Util  Compute M. |
|               |                    MIG M. |
+-----+-----+
| 0  NVIDIA GeForce ... Off | 00000000:0A:00:0 Off |           N/A       |
| 30%  25C   P8   26W / 350W | 6424MiB / 24576MiB |      0%   Default   |
|                               |                       |                       |
| 1  NVIDIA GeForce ... Off | 00000000:0B:00:0 Off |           N/A       |
| 0%   32C   P8   29W / 390W | 7116MiB / 24576MiB |      0%   Default   |
|                               |                       |                       |
+-----+-----+
Processes:
+-----+-----+
| GPU  GI  CI       PID   Type   Process name                      GPU Memory |
|   ID   ID                                     Usage      |
+-----+-----+
| 0  N/A  N/A    1395   G   /usr/lib/xorg/Xorg                49MiB   |
| 0  N/A  N/A    2886   G   /usr/lib/xorg/Xorg                42MiB   |
| 0  N/A  N/A    31139  C   python                           6328MiB |
| 1  N/A  N/A    1395   G   /usr/lib/xorg/Xorg                4MiB    |
| 1  N/A  N/A    31139  C   python                           7108MiB |
+-----+-----+

```

这里输入 `Stop` 退出启动程序，GPU资源就会立即被释放。

root@ff: /home/Work/00.Work/mymu_yuanyu_chatglm3_6b/ChatGLM3/... x

every 1.0s: nvidia-smi ff: Tue Jan 9 11:35:23 2024

```

+-----+
| NVIDIA-SMI 525.116.03   | Driver Version: 525.116.03   | CUDA Version: 12.0   |
+-----+-----+
| GPU Name      Persistence-M| Bus-Id        Disp.A    Volatile Uncorr. ECC  |
| Fan  Temp  Perf  Pwr:Usage/Cap|  Memory-Usage  GPU-Util  Compute M.     |
|================================+==+
| 0  NVIDIA GeForce ...  Off  | 00000000:0A:00:00 Off  | 11%  Default  N/A   |
| 30%  25C   P8     26W / 350W | 94MiB / 24576MiB      |              N/A   |
+-----+-----+
| 1  NVIDIA GeForce ...  Off  | 00000000:0B:00:00 Off  | 0%    Default  N/A   |
| 0%   32C   P8     28W / 390W | 6MiB / 24576MiB      |              N/A   |
+-----+-----+

```

用户 **stop** 输入Stop 退出后，在观察GPU状态

root@ff: /home/Work/00.Work/mymu_yuanyu_chatglm3_6b/ChatGLM3/... x

```

+-----+
| Processes: |
| GPU   GI   CI   PID    Type    Process name          | GPU Memory |
| ID    ID    ID           |            | Usage         |
+-----+-----+
| 0  N/A  N/A   1395    G    /usr/lib/xorg/Xorg    | 49MiB      |
| 0  N/A  N/A   2086    G    /usr/lib/xorg/Xorg    | 43MiB      |
| 1  N/A  N/A   1395    G    /usr/lib/xorg/Xorg    | 4MiB       |
+-----+-----+

```

显示占用已降低，且没有ChatGLM3-6B项目运行的进程

默认启动会自动使用多块GPU的资源的原因，在于 `cli_demo.py` 这个.py文件中的这行代码：

```
(chatglm3_multi) root@ff:basic_demo# pwd
/home/Work/00.Work_muyu/muyu_chatglm3_6b/chatGLM3/basic_demo
(chatglm3_multi) root@ff:basic_demo# vim cli_demo.py
```

参数 `device_map="auto"`，这个参数指示 transformers 库自动检测可用的 GPU 并将模型的不同部分映射到这些 GPU 上。如果机器上有多个 GPU，模型会尝试在这些 GPU 上进行分布式处理。其通过分析各个 GPU 的当前负载和能力来完成。负载均衡的目标是最大化所有 GPU 的利用率，避免任何一个 GPU 过载。

```
from transformers import AutoTokenizer, AutoModel
#MODEL_PATH = os.environ.get('MODEL_PATH', 'THUDM/chatglm3-6b')
MODEL_PATH = os.environ.get('MODEL_PATH', './chatglm3-6b')
TOKENIZER_PATH = os.environ.get("TOKENIZER_PATH", MODEL_PATH)

tokenizer = AutoTokenizer.from_pretrained(TOKENIZER_PATH, trust_remote_code=True)
model = AutoModel.from_pretrained(MODEL_PATH, trust_remote_code=True, device_map="auto").eval()

os_name = platform.system()
```

可以通过如下代码，查看当前环境下的 GPU 情况：

```
import torch

# 检查 CUDA 是否可用
cuda_available = torch.cuda.is_available()
print(f"CUDA available: {cuda_available}")

# 列出所有可用的 GPU
if cuda_available:
    num_gpus = torch.cuda.device_count()
    print(f"Number of GPUs available: {num_gpus}")

    for i in range(num_gpus):
        print(f"GPU {i}: {torch.cuda.get_device_name(i)}")

# 获取当前默认 GPU
print(f"Current CUDA device: {torch.cuda.current_device()}")
else:
    print("No GPUs available.")
```

可以把上述代码写在一个 .py 文件中，执行该文件后会输出当前机器上的 GPU 资源情况，方便我们对当前的资源情况有一个比较清晰的认知。

```
(chatglm3_multi) root@ff:basic_demo# vim gpu_check.py
(chatglm3_multi) root@ff:basic_demo# python gpu_check.py
CUDA available: True
Number of GPUs available: 2
GPU 0: NVIDIA GeForce RTX 3090
GPU 1: NVIDIA GeForce RTX 3090
Current CUDA device: 0
```

将上述代码写在该文件中

如果想要指定使用某一块 GPU，那么需要这样修改代码 `cli_demo.py` 中的代码：


```
import torch

# 设置 GPU 设备
device = torch.device('cuda:0' if torch.cuda.is_available() else 'cpu')

#model = AutoModel.from_pretrained(MODEL_PATH, trust_remote_code=True,
device_map="auto").eval()
model = AutoModel.from_pretrained(MODEL_PATH, trust_remote_code=True).eval()

# 将模型移到指定的 GPU
model = model.to(device)
```

```
import os
import platform
from transformers import AutoTokenizer, AutoModel
import torch  → 新增
#MODEL_PATH = os.environ.get('MODEL_PATH', 'THUDM/chatglm3-6b')
MODEL_PATH = os.environ.get('MODEL_PATH', '../chatglm3-6b')
TOKENIZER_PATH = os.environ.get("TOKENIZER_PATH", MODEL_PATH)

tokenizer = AutoTokenizer.from_pretrained(TOKENIZER_PATH, trust_remote_code=True)

# 设置 GPU 设备
device = torch.device('cuda:0' if torch.cuda.is_available() else 'cpu')  → 新增

#model = AutoModel.from_pretrained(MODEL_PATH, trust_remote_code=True, device_map="auto").eval()
model = AutoModel.from_pretrained(MODEL_PATH, trust_remote_code=True).eval()  → 修改

# 将模型移到指定的 GPU
model = model.to(device)  → 新增

os_name = platform.system()
clear_command = 'cls' if os_name == 'Windows' else 'clear'
stop_stream = False
```

修改后看下启动情况：

```
Every 1.0s: nvidia-smi
Tue Jan 9 14:11:20 2024
```

GPU	Name	Persistence-M	Bus-Id	Disp.A	Volatile Uncorr. ECC
0	NVIDIA GeForce ...	Off	80000000:0A:00:0	Off	N/A
1	NVIDIA GeForce ...	Off	80000000:0B:00:0	Off	N/A

```
Processes:
GPU  GI  CI  PID  Type  Process name  GPU Memory
Usage
0  N/A  N/A  1395  G  /usr/lib/xorg/Xorg  49MB
0  N/A  N/A  2086  G  /usr/lib/xorg/Xorg  42MB
0  N/A  N/A  54273  C  python  12686MB
1  N/A  N/A  1395  G  /usr/lib/xorg/Xorg  4MB
```

```
(chatglm3_multi) root@ff:basic_demo# vim cli_demo.py
(chatglm3_multi) root@ff:basic_demo# python cli_demo.py
Loading checkpoint shards: 100%
欢迎使用 ChatGLM3-6B 模型，输入内容即可进行对话，clear 清空对话历史，stop 终止程序
用户：
```

指定第0块GPU启动程序

在代码程序中指定某几块GPU加载服务

更多人的情况是：比如当前机器中有4块GPU，我们只想使用前两块GPU做此次任务的加载，该如何选择呢？这很常见，其问题主要在于：如果某块GPU已经处于满载运行当中，这时我们再使用四块默认同时运行的话大概率会提示out of memory报错，或者提示显卡不平衡imblance的warning警告。

如果是想在代码中指定多块卡运行该服务，需要在代码中添加这两行代码：

```
import os
os.environ["CUDA_VISIBLE_DEVICES"] = ','.join(map(str, [0,1]))
```



```

import os
import platform
from transformers import AutoTokenizer, AutoModel
import torch

os.environ["CUDA_VISIBLE_DEVICES"] = ','.join(map(str, [0,1]))
#MODEL_PATH = os.environ.get('MODEL_PATH', 'THUDM/chatglm3-6b')
MODEL_PATH = os.environ.get('MODEL_PATH', './chatglm3-6b')
TOKENIZER_PATH = os.environ.get('TOKENIZER_PATH', MODEL_PATH)

tokenizer = AutoTokenizer.from_pretrained(TOKENIZER_PATH, trust_remote_code=True)

model = AutoModel.from_pretrained(MODEL_PATH, trust_remote_code=True, device_map="auto").eval()

```

→ 这里的[0,1]就是显卡的标识

然后保存修改后，执行启动过程就可以了。

```

Every 1.0s: nvidia-smi
Thu Jan 11 12:05:03 2024
+-----+
| NVIDIA-SMI 525.116.03   Driver Version: 525.116.03   CUDA Version: 12.0   |
+-----+-----+-----+-----+-----+-----+
| GPU Name      Persistence-M| Bus-Id        Disp.A | Volatile Uncorr. ECC |
| Fan  Temp  Perf  Pwr:Usage/Cap|  Memory-Usage | GPU-Util  Compute M. |
|-----+-----+-----+-----+-----+-----+
| 0  NVIDIA GeForce ...  Off | 00000000:0A:00:0 Off |   N/A   Default      N/A   |
| 30%  36C  P2    204W / 350W | 6784MiB / 24576MiB |   47%    Default      N/A   |
+-----+-----+-----+-----+-----+-----+
| 1  NVIDIA GeForce ...  Off | 00000000:0B:00:0 Off |   N/A   Default      N/A   |
| 30%  48C  P2    234W / 390W | 7500MiB / 24576MiB |   47%    Default      N/A   |
+-----+-----+-----+-----+-----+-----+

```

双卡加载

```

(chatglm3 multi) root@ff:basic_demo# vim cli_demo.py
(chatglm3 multi) root@ff:basic_demo# python cli_demo.py
Loading checkpoint shards: 100%
欢迎使用 ChatGLM3-6B 模型，输入内容即可进行对话，clear 清空对话历史，stop 终止程
用户：
ChatGLM:

```

启动服务

• 直接使用CUDA_VISIBLE_DEVICES环境变量启动

第二种方法就是设置 CUDA 设备环境变量。这个方法非常简单，且不涉及更改Python代码。只需要在运行 Python 脚本之前，在命令行中设置 CUDA_VISIBLE_DEVICES 环境变量。这个环境变量告诉 PyTorch 使用哪个 GPU。例如，如果想使用第二块 GPU（GPU 编号从 0 开始，因此第二块 GPU 是 1），就可以这样启动程序：

```

Every 1.0s: nvidia-smi
Tue Jan 9 14:47:15 2024
+-----+
| NVIDIA-SMI 525.116.03   Driver Version: 525.116.03   CUDA Version: 12.0   |
+-----+-----+-----+-----+-----+-----+
| GPU Name      Persistence-M| Bus-Id        Disp.A | Volatile Uncorr. ECC |
| Fan  Temp  Perf  Pwr:Usage/Cap|  Memory-Usage | GPU-Util  Compute M. |
|-----+-----+-----+-----+-----+-----+
| 0  NVIDIA GeForce ...  Off | 00000000:0A:00:0 Off |   N/A   Default      N/A   |
| 30%  29C  P8    26W / 350W | 90MiB / 24576MiB |   11%    Default      N/A   |
+-----+-----+-----+-----+-----+-----+
| 1  NVIDIA GeForce ...  Off | 00000000:0B:00:0 Off |   N/A   Default      N/A   |
| 0%  36C  P8    35W / 390W | 12694MiB / 24576MiB |    0%    Default      N/A   |
+-----+-----+-----+-----+-----+-----+

```

设置只使用第1块GPU

```

Processes:
GPU  GI  CI  PID  Type  Process name      GPU Memory
ID   ID
-----
0  N/A  N/A  1395  G    /usr/lib/xorg/Xorg  49MiB
0  N/A  N/A  2086  G    /usr/lib/xorg/Xorg  38MiB
1  N/A  N/A  1395  G    /usr/lib/xorg/Xorg  4MiB
1  N/A  N/A  57636 C    python              12686MiB

```

如果想使用两块 GPU启动，那么可以使用逗号（，）来进行分割。

```

root@ff: /home/Work/00.Work_muyu/muyu_chatglm3_6b/ChatGLM3/ba...
Every 1.0s: nvidia-smi
Tue Jan 9 14:52:12 2024
+-----+
| GPU Name      Persistence-M| Bus-Id        Disp.A | Volatile Uncorr. ECC |
| Fan  Temp  Perf  Pwr:Usage/Cap|  Memory-Usage | GPU-Util  Compute M. |
|-----+-----+-----+-----+-----+-----+
| 0  NVIDIA GeForce ...  Off | 00000000:0A:00:0 Off |   N/A   Default      N/A   |
| 30%  29C  P8    26W / 350W | 6424MiB / 24576MiB |    0%    Default      N/A   |
+-----+-----+-----+-----+-----+-----+
| 1  NVIDIA GeForce ...  Off | 00000000:0B:00:0 Off |   N/A   Default      N/A   |
| 0%  49C  P2   118W / 390W | 7116MiB / 24576MiB |    0%    Default      N/A   |
+-----+-----+-----+-----+-----+-----+

```

使用第0块和第1块GPU来启动程序

```

Processes:
GPU  GI  CI  PID  Type  Process name      GPU Memory
ID   ID
-----
0  N/A  N/A  1395  G    /usr/lib/xorg/Xorg  49MiB
0  N/A  N/A  2086  G    /usr/lib/xorg/Xorg  42MiB
0  N/A  N/A  58448 C    python              6328MiB
1  N/A  N/A  1395  G    /usr/lib/xorg/Xorg  4MiB
1  N/A  N/A  58448 C    python              7108MiB

```

同时，在执行推理的过程中，其功率也会增长。

NVIDIA-SMI 525.116.03 Driver Version: 525.116.03 CUDA Version: 12.0									
GPU	Name	Persistence-M	Bus-Id	Disp.A	Volatile Uncorr. ECC	GPU-Util	Compute M.	MIG M.	
Fan	Temp	Perf	Pwr:Usage/Cap	Memory-Usage					
0	NVIDIA GeForce ...	Off	00000000:0A:00.0	Off		78%	Default	N/A	
81%	63C	P2	297W / 350W	4832MiB / 24576MiB					
1	NVIDIA GeForce ...	Off	00000000:0B:00.0	Off		0%	Default	N/A	
0%	34C	P8	28W / 390W	8MiB / 24576MiB					
Processes:									
GPU	GI	CI	PID	Type	Process name	GPU Memory			
ID	ID	ID							

The algorithms used in machine learning are designed for supervised learning tasks, such as classification or predictions. These algorithms can be supervised by providing them with labeled data to learn from.

Machine learning can be used in a wide variety of applications, including recommendation systems, image recognition, and predictive modeling. Some of the most common machine learning algorithms include linear regression, decision trees, and support vector machines.

In summary, machine learning is a powerful tool for analyzing data and making predictions. It has numerous applications across various industries, and algorithms are developed to solve specific problems.

用户: exit

ChatGLM: Sure, feel free to ask if you have any questions.

用户: **stop** → **输入Stop退出**
(chatglm3_multi) root@ff:~# basic_demo#

3.ChatGLM3-6B 高效微调实践

在大模型掀起新一轮的AI热潮以来，目前的形式就是大语言模型（LLM）百花齐放，工业界用于生产的算法模型由原来是几万，几十万的参数，到现在上升到上百亿，上百亿的情况。在这种情况下，因为显卡资源的因素，预训练大模型基本是大公司或者高校才可以做的事情，小公司或个人只能对大模型进行微调后使用。

以前我们比较熟悉的都是全量微调，这个微调过程是对原始模型的所有参数全部做一个调整。但对于LLM，在消费级显卡上就做根本没有办法实现。所以目前对于大模型来说，主流的微调技术叫做高效微调，这种方式是通过微调大模型少量或者额外的一些参数，固定预训练模型（LLM）参数，以此来降低计算和存储成本，同时，还可以在在一定程度上实现与全量参数微调相当的性能。

3.1 主流的高效微调方法介绍

- Freeze

Freeze是冻结的意思，Freeze方法指的是参数冻结，对原始模型的大部分参数进行冻结，仅训练少部分的参数，这样就可以大大减少显存的占用，从而完成对大模型的微调。特别是在Bert模型出来的时候，比较会常用到Freeze的这样一个微调方法，比如Bert有12层，我们把前10层冻结了，只训练后两层。这是一种比较简单微调方法，由于冻结的参数是大部分，微调的参数是少部分，因此在代码中只需要设置需要微调的层的参数即可，把不需要参加训练的层数 `requires_grad` 设置为False，不对其进行更新，从而达到冻结的这样一个效果。

```
for name, param in model.named_parameters():
    if not any(nd in name for nd in ["layers.27", "layers.26", "layers.25", "layers.24", "layers.23"]):
        param.requires_grad = False
```

- Prefix-Tuning (2021年提出)

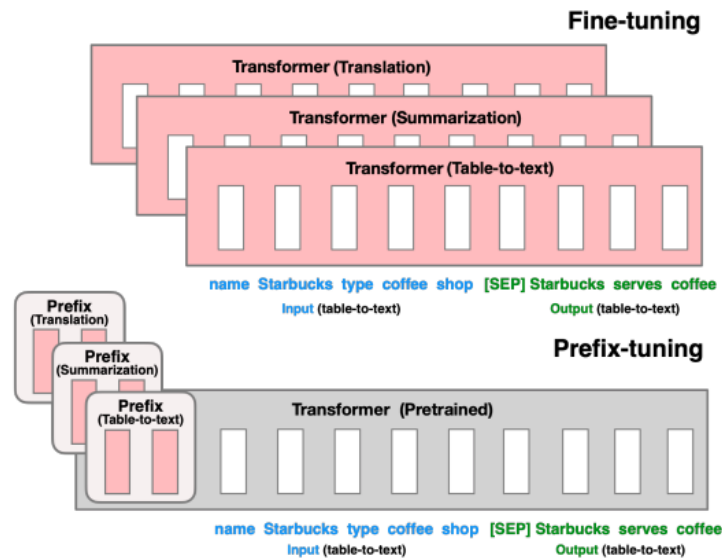
Prefix-Tuning指的是在微调模型的过程中只优化加入的一小段可学习的向量(virtual tokens)作为Prefix，而不需要优化整个模型的参数（训练的时候只更新Prefix部分的参数，而PLM中的其他部分参数固定）。

Prefix-Tuning论文地址: <https://arxiv.org/abs/2101.00190>

Prefix-Tuning代码地址: <https://github.com/XiangLi1999/PrefixTuning>

传统的微调范式Fine-tuning会利用预训练模型去对不同的下游任务进行微调，对每个任务都要保存一份微调后的模型权重。比如下图展示的三个不同任务的Transformer模型，分别用来做翻译、摘要和将格式转化（table-to-text）。每个任务都有自己的微调模型，这意味着模型的所有权重都在微调过程中针对特定任务进行了更新。这种方法通常需要大量的数据和计算资源，因为整个模型都在学习任务特定的知识。

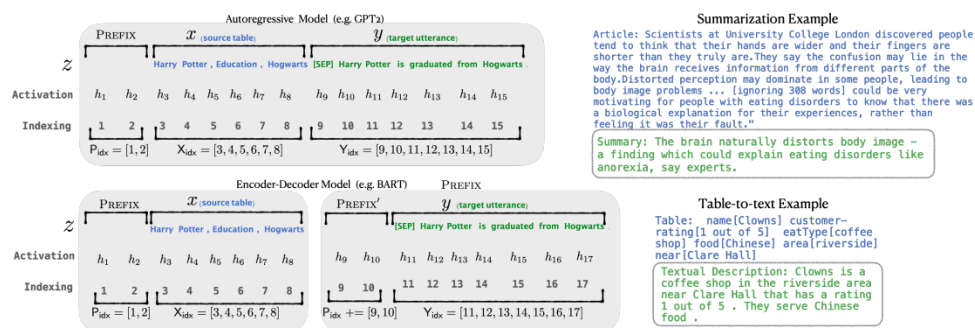
Prefix-tuning 就提出了一种不同的微调策略，对基于Transformers结构的模型，它会将特定的前缀添加到输入序列的开始部分，相当于任务特定的提示，可以是一组固定的词或是可训练的嵌入向量。



但是这个Prefix 并不是一些明确的单词，比如对于文本摘要任务来说，我们添加 this is summarization（明确指出这是一个摘要的任务），相反，这个prefix加的是一些隐式的Token。这里就需要了解两个概念：

- Hard Prompt：也称离散Prompt，是一个实际的文本字符串（自然语言，人工可读），通常由中文或英文词汇组成；
- Soft Prompt：也称连续Prompt，通常是在向量空间优化出来的提示，通过梯度搜索之类的方式进行优化；

在Hoft Promot中，提示语的变化对模型最终的性能特别敏感，加一个词、少一个词或者变动位置都会造成比较大的变化。成本比较高，并且效果不太好。显然：Prefix Tuning属于Soft prompt。也就是我们学习调整的就是这部分的参数，从而达到微调的目的。



Encoder端增加前缀是为了引导输入部分的编码，Decoder 端增加前缀是为了引导后续token的生成。

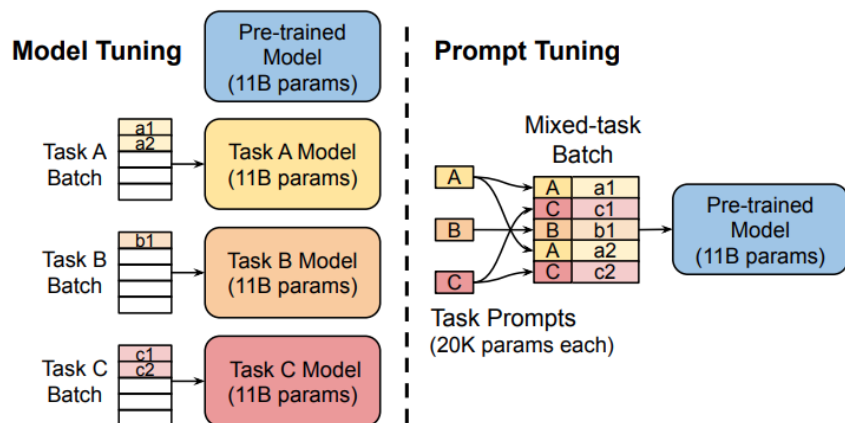
Prefix-tuning 的优势在于它不需要调整模型的全部权重，而是通过在输入中添加前缀来调整模型的行为，这样可以节省大量的计算资源，同时使得一个单一模型能够适应多种不同的任务。

• Prompt Tuning (2021年提出)

Prompt Tuning 方法可以看做是Prefix Tuning的简化版本，它给每个任务都定义了自己的Prompt，将真实的Tokens转化为可微的virtual token，并加入人工设计的锚字符（与任务高度相关的Token），拼接到数据上作为输出，但只在输入层加入Prompt tokens。

Prompt Tuning论文地址：<https://arxiv.org/pdf/2104.08691.pdf>

如图所示：如果A、B、C三个任务，模型框架都是一样的，对每一个任务都添加一个自己定义的Prompt，然后把每个任务混合在一起放在一个Batch中来进行训练。



下面的训练例子说明了两者的区别：

Prompt Tuning 示例：

输入序列： "Prompt 1, Prompt2 | 这部电影令人振奋。"

问题：评价这部电影的情感倾向。

答案：模型需要预测情感倾向（例如“积极”）

提示：无明确的外部提示，

充当引导模型的内部提示，因为这里的问题是隐含的，即判断文本中表达的情感倾向。

Prefix Tuning 示例：

输入序列： " Prefix1, Prefix 2 | I want to watch a movie."

问题：根据前缀生成后续的自然语言文本。

答案：模型生成的文本，如“that is exciting and fun.”

提示：前缀本身提供上下文信息，没有单独的外部提示

所以Prompt Tuning和Prefix Tuning都涉及在输入数据中加入可学习的向量，但两者的策略和目的不一样：

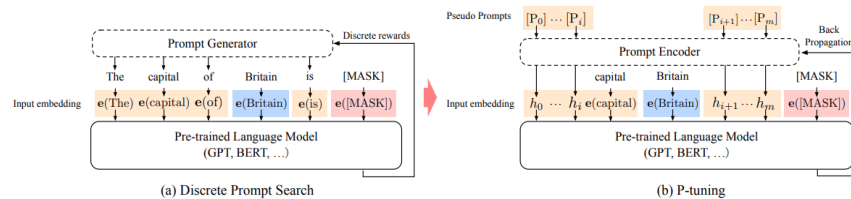
- **Prompt Tuning:** 可学习向量（通常称为prompt tokens）旨在模仿自然语言提示的形式，它们被设计为引导模型针对特定任务生成特定类型的输出。这些向量通常被看作是任务指导信息的一部分，倾向于用更少量的向量模仿传统的自然语言提示。
- **Prefix Tuning:** 可学习前缀Prefix更多地用于提供输入数据的直接上下文信息，这些前缀作为模型内部表示的一部分，可以影响整个模型的行为。
- **P-Tuning v1**

P-Tuning V1的核心是使用可微的virtual token替换了原来的discrete tokens，且仅加入到输入层，并使用prompt encoder（BiLSTM+MLP）对virtual token进行编码学习。

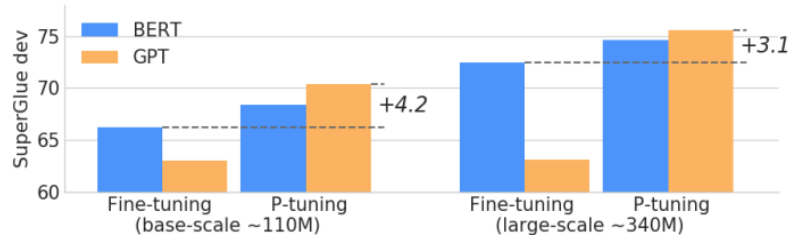
P-Tuning v1 论文地址：<https://arxiv.org/pdf/2103.10385.pdf>

P-tuning v1 github 代码: <https://github.com/THUDM/P-tuning>

Prompt Tuning会使用静态的、可训练的虚拟标记嵌入。这些嵌入在初始化后保持固定, 除非在训练过程中被更新, 相对简单, 因为它只涉及调整一组固定的嵌入参数。在处理多种任务时表现良好, 但在处理特别复杂或需要细粒度控制的任务时受限。所以, P-Tuning v1 就在输入的句子中也是加入了隐式的 virtual token, 区别就是: 前面的方式是直接对它进行一个学习更新, 只不过不会更新大模型中的参数, 只是更新我们加入的 virtual token这样一个参数, P-Tuning v1 是对添加的virtual Token, 又使用BiLSTM + MLP 对其进行了一个编码。



虽然这个编码对于PLM来说简单多了, 参数也都非常小。但是, 也能起到一个比较好的效果。相同参数规模, 如果进行全参数微调, Bert在NLU任务上的效果, 超过GPT很多; 但是在P-Tuning下, GPT可以取得超越Bert的效果。



那么Prompt Tuning和P-Tuning等方法存在两个主要的问题:

- 缺乏模型参数规模和任务通用性: Prompt Tuning论文中表明当模型规模超过100亿个参数时, 提示优化可以与全量微调相媲美。但是对于那些较小的模型 (从100M到1B), 提示优化和全量微调的表现有很大差异, 这大大限制了提示优化的适用性。
- 缺乏任务普遍性: 尽管Prompt Tuning和P-tuning在一些 NLU 基准测试中表现出优势, 但提示调优对硬序列标记任务 (即序列标注) 的有效性尚未得到验证。
- 缺少深度提示优化, 在Prompt Tuning和P-tuning中, 连续提示只被插入transformer第一层的输入 embedding序列中, 在接下来的transformer层中, 插入连续提示的位置的embedding是由之前的transformer层计算出来的, 这可能导致两个可能的优化挑战。

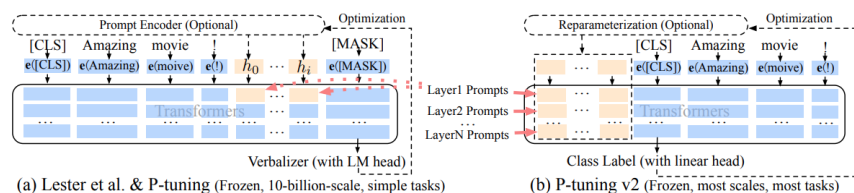
由于序列长度的限制, 可调参数的数量是有限的, 输入embedding对模型预测只有相对间接的影响。这些问题在P-tuning v2得到了改进。

• P-Tuning v2

P-Tuning v2主要是基于P-tuning和Prefix-tuning技术, 最核心的是引入Deep Prompt Encoding和Multi-task Learning等策略进行优化的。

P-Tuning v2论文地址: <https://arxiv.org/abs/2110.07602>

P-Tuning v2 github代码: <https://github.com/THUDM/P-tuning-v2>



Deep Prompt Encoding: P-Tuning v2在每一层都加入了Prompts tokens作为输入，而不是仅仅加在输入层，这带来两个方面的好处：

- 更多可学习的参数（从P-tuning和Prompt Tuning的0.01%增加到0.1%-3%），同时也足够参数高效。
- 加入到更深层结构中的Prompt能给模型预测带来更直接的影响。

Multi-task learning：基于多任务数据集的Prompt进行预训练，然后再适配到下游任务。对于pseudo token的continuous prompt，随机初始化比较难以优化，因此采用multi-task方法同时训练多个数据集，共享continuous prompts去进行多任务预训练，可以让prompt有比较好的初始化。

所以P-Tuning v2是一种在不同规模和任务中都可与微调相媲美的提示方法。P-Tuning v2对从330M到10B的模型显示出一致的改进，并在序列标注等困难的序列任务上以很大的幅度超过了Prompt Tuning和P-Tuning。

除此之外，还有比较主流的LoRA，QLoRA，感兴趣的也可以自行了解一下。本篇内容主要涉及ChatGLM3-6B模型的P-Tuning V2 高效微调。

3.2 ChatGLM3-6B模型的高效微调实践

本次实验环境配置1：

- 操作系统：Ubuntu 22.04；
- GPU：3090双卡，总共48G显存；
- CPU：AMD 5900X；
- 存储：64G内存+2T SSD数据盘；

实验环境配置2：

- 操作系统：CentOs 7.3；
- GPU：4090双卡，总共48G显存；
- CPU：24 vCPU Intel(R) Xeon(R) Platinum 8352V CPU
- 存储：180GB + 100G数据盘

ChatGLM官网出了一个基于P-Tuning v2的方式微调ChatGLM-6B的项目，项目地址：<https://github.com/THUDM/ChatGLM-6B/tree/main/ptuning>，最低只需要 7GB 显存即可运行。

量化等级	最低 GPU 显存 (推理)	最低 GPU 显存 (高效参数微调)
FP16 (无量化)	13 GB	14 GB
INT8	8 GB	9 GB
INT4	6 GB	7 GB

在ChatGLM3-6B模型的项目文件中，官方提供了基于ChatGLM3-6B-base模型和ChatGLM3-6B两个基座模型的微调示例，其中ChatGLM3-6B-base模型仅提供了Lora微调，而ChatGLM3-6B包括全量微调和P-Tuning V2。相关存储位置如下：

```
total 124
drwxr-xr-x 15 root root 4096 Jan 10 16:41 ./
drwxr-xr-x 3 root root 30 Jan 10 16:39 ../
drwxr-xr-x 8 root root 4096 Jan 10 16:39 .git/
drwxr-xr-x 4 root root 69 Jan 10 16:39 .github/
-rw-r--r-- 1 root root 175 Jan 10 16:39 .gitignore
-rw-r--r-- 1 root root 2098 Jan 10 16:39 DEPLOYMENT.md
-rw-r--r-- 1 root root 2304 Jan 10 16:39 DEPLOYMENT_en.md
-rw-r--r-- 1 root root 11353 Jan 10 16:39 LICENSE
-rw-r--r-- 1 root root 4133 Jan 10 16:39 MODEL_LICENSE
-rw-r--r-- 1 root root 6885 Jan 10 16:39 PROMPT.md
-rw-r--r-- 1 root root 7118 Jan 10 16:39 PROMPT_en.md
-rw-r--r-- 1 root root 17114 Jan 10 16:39 README.md
-rw-r--r-- 1 root root 17801 Jan 10 16:39 README_en.md
drwxr-xr-x 2 root root 169 Jan 10 16:39 basic_demo/
drwxr-xr-x 2 root root 4096 Jan 10 16:39 chatglm3-6b/
drwxr-xr-x 4 root root 4096 Jan 10 16:39 composite_demo/
drwxr-xr-x 3 root root 104 Jan 10 16:39 cookbook/
drwxr-xr-x 3 root root 4096 Jan 10 16:39 finetune_base_model_demo/
drwxr-xr-x 6 root root 4096 Jan 10 16:45 finetune_chatmodel_demo/
drwxr-xr-x 3 root root 53 Jan 10 16:39 langchain_demo/
drwxr-xr-x 2 root root 110 Jan 10 16:39 openai_api_demo/
-rw-r--r-- 1 root root 474 Jan 10 16:39 requirements.txt
drwxr-xr-x 2 root root 4096 Jan 10 16:39 resources/
drwxr-xr-x 2 root root 55 Jan 10 16:39 tensorrt_llm_demo/
drwxr-xr-x 2 root root 117 Jan 10 16:39 tools_using_demo/
-rw-r--r-- 1 root root 152 Jan 10 16:39 update_requirements.sh
```

微调ChatGLM3-6B-base模型的
微调示例，仅包含Lora

微调ChatGLM3-6B模型的微调示例
包含全量微调和P-Tuning V2

base模型不具备对话能力，仅能够生成单轮回复。如果大家希望使用多轮对话模型，需要对Chat模型进行微调，所以需要用到 `finetune_chatmodel_demo` 下的参考代码，进入后，相关的项目代码如下：

```
(chatglm3_multi) root@autodl-container-fff445a19e-a56aeebe:~/autodl-tmp/muyu_chatglm3/ChatGLM3# cd finetune_chatmodel_demo/
(chatglm3_multi) root@autodl-container-fff445a19e-a56aeebe:~/autodl-tmp/muyu_chatglm3/ChatGLM3/finetune_chatmodel_demo# ll
total 60
drwxr-xr-x 5 root root 4096 Jan 10 16:54 ./
drwxr-xr-x 16 root root 4096 Jan 10 16:54 ../
-rw-r--r-- 1 root root 8753 Jan 10 16:39 README.md
drwxr-xr-x 2 root root 126 Jan 10 16:45 _pycache_/
-rw-r--r-- 1 root root 4997 Jan 10 16:39 arguments.py
drwxr-xr-x 2 root root 36 Jan 10 16:39 configs/
-rw-r--r-- 1 root root 7083 Jan 10 16:39 finetune.py
-rw-r--r-- 1 root root 1986 Jan 10 16:39 inference.py
-rw-r--r-- 1 root root 6252 Jan 10 16:39 preprocess_utils.py
-rw-r--r-- 1 root root 114 Jan 10 16:39 requirements.txt
drwxr-xr-x 3 root root 4096 Jan 10 16:48 scripts/
-rw-r--r-- 1 root root 3279 Jan 10 16:39 trainer.py
```

微调的说明文件

模型参数配置文件

DeepSpeed的配置文件

接口文件（主程序）

推理接口文件

逻辑处理代码

依赖环境

微调启动脚本

模型训练文件

无论是全量微调还是P-Tuning v2，都需要设计微调数据，ChatGLM3-6B支持多轮对话和输入输出格式微调样例。因此如果想要使用自己的数据集进行模型微调，需要首先统一样例格式。同时，ChatGLM3-6B微调对话和微调工具能力的数据格式也不相同。

这里我们启动微调的脚本存放在 `script` 文件目录下。

```
total 92
drwxr-xr-x 2 root root 4096 Jan 11 13:41 ./
drwxr-xr-x 4 root root 4096 Jan 11 13:41 ../
-rw-r--r-- 1 root root 1153 Jan 11 13:41 finetune_ds.sh
-rw-r--r-- 1 root root 1010 Jan 11 13:41 finetune_ds_multiturn.sh
-rw-r--r-- 1 root root 1124 Jan 11 13:41 finetune_pt.sh
-rw-r--r-- 1 root root 1048 Jan 11 13:41 finetune_pt_multiturn.sh
-rw-r--r-- 1 root root 585 Jan 11 13:41 format_advertise_gen.py
-rw-r--r-- 1 root root 1822 Jan 11 13:41 format_tool_alpaca.py
```

单轮对话的全量微调脚本

多轮对话的全量微调脚本

单轮对话的P-Tuning v2 微调脚本

多轮对话的P-Tuning V2 微调脚本

格式化单轮对话Advertise数据集

格式化多轮对话ToolAlpaca数据集

• 单轮对话微调

首先来看单轮对话微调，对于输入-输出格式，样例采用如下输入格式：

```
[
{
    "prompt": "<prompt text>",
    "response": "<response text>"
}
// ...
]
```

官网提供了一个微调示例：AdvertiseGen 数据集，可以进入Tsinghua Cloud: <https://cloud.tsinghua.edu.cn/f/b3f119a008264b1cabd1/?dl=1> 下载并上传到 finetune_chatmodel_demo 路径下。一种更便捷的方式就是在服务器终端使用 wget 命令来进行下载。同时下载到的AdvertiseGen数据集是一个.tar.gz的压缩文件，需要解压才可使用：

```
wget -O AdvertiseGen https://cloud.tsinghua.edu.cn/f/b3f119a008264b1cabd1/?dl=1
```

```
(chatglm3_multi) root@autodl-container-9052489e7e-Sac20027:/opt/muyu_chatglm3_6b/ChatGLM3/finetune_chatmodel_demo/scripts# wget https://cloud.tsinghua.edu.cn/f/b3f119a008264b1cabd1/?dl=1
--2024-01-10 12:56:21-- https://cloud.tsinghua.edu.cn/f/b3f119a008264b1cabd1/?dl=1
Resolving cloud.tsinghua.edu.cn (cloud.tsinghua.edu.cn)... 166.111.6.101. 2402:f000:1:406:166:111:6:101
Connecting to cloud.tsinghua.edu.cn (cloud.tsinghua.edu.cn)|166.111.6.101|:443... connected.
HTTP request sent, awaiting response... 302 Found
Location: https://cloud.tsinghua.edu.cn/seahttp/files/85ced2dc-eab3-4c29-ab0-e68f30ea7c32/AdvertiseGen.tar.gz [following]
--2024-01-10 12:56:21-- https://cloud.tsinghua.edu.cn/seahttp/files/85ced2dc-eab3-4c29-ab0-e68f30ea7c32/AdvertiseGen.tar.gz
Resolving existing connection to cloud.tsinghua.edu.cn:443.
HTTP request sent, awaiting response... 200 OK
Length: 17069994 (16M) [application/octet-stream]
Saving to: 'index.html?dl=1'

index.html?dl=1                                100%[=====>] 16.28M  26.5MB/s  in
2024-01-10 12:56:22 (26.5 MB/s) - 'index.html?dl=1' saved [17069994/17069994]

(chatglm3_multi) root@autodl-container-9052489e7e-Sac20027:/opt/muyu_chatglm3_6b/ChatGLM3/finetune_chatmodel_demo/scripts# ll
total 16784
drwxr-xr-x 3 root root 4096 Jan 10 12:56 ./
drwxr-xr-x 5 root root 4096 Jan 10 12:50 ../
-rw-r--r-- 1 root root 1153 Jan 10 12:11 finetune_ds.sh
-rw-r--r-- 1 root root 1010 Jan 10 12:11 finetune_ds_multiturn.sh
-rw-r--r-- 1 root root 1124 Jan 10 12:11 finetune_pt.sh
-rw-r--r-- 1 root root 1048 Jan 10 12:11 finetune_pt_multiturn.sh
-rw-r--r-- 1 root root 585 Jan 10 12:11 format_advertise_gen.py
-rw-r--r-- 1 root root 1822 Jan 10 12:11 format_tool_alpaca.py
drwxr-xr-x 2 root root 59 Jan 10 12:52 formatted_data/
-rw-r--r-- 1 root root 17069994 Jan 10 12:56 index.html?dl=1
(chatglm3_multi) root@autodl-container-9052489e7e-Sac20027:/opt/muyu_chatglm3_6b/ChatGLM3/finetune_chatmodel_demo/scripts# tar -xvf 'index.html?dl=1'
AdvertiseGen/
AdvertiseGen/train.json
AdvertiseGen/dev.json
(chatglm3_multi) root@autodl-container-9052489e7e-Sac20027:/opt/muyu_chatglm3_6b/ChatGLM3/finetune_chatmodel_demo/scripts#
```

ADGEN 数据集任务为根据输入 (content) 生成一段广告词 (summary)，其数据格式如下：

```
{"content": "类型#上衣*版型#宽松*版型#显瘦*衣样式#外套*衣袖型#插肩袖*衣款式#拼接", "summary": "宽松的版型，穿搭起来总是不挑身材；所以作为早春穿搭的话，有着插肩袖设计的这款外套，最能展现出舒适和大方的感觉了。而比较宽松的外套款式，它在衣身上特别做了拼接的设计，你看那颜色独特的拼接，很容易就能展现出独特和抢眼的效果；再加上直筒的版型设计，穿搭起来真的是一点也不挑身材，还能起到显瘦的效果。"}

```

```
{"content": "类型#上衣*风格#运动*风格#休闲*衣样式#外套*衣领型#立领*衣袖长#长袖*衣门襟#拉链*衣款式#拉链", "summary": "基础的外套廓形，直筒，立领长袖，中间金属拉链穿脱，方便实用，带有浓浓的休闲运动味道。日常休闲上班或是去<UNK>等运动时都可以穿着，时尚前卫。"}

```

```
{"content": "类型#上衣*风格#街头*图案#创意*衣样式#卫衣", "summary": "在这件卫衣上，BRAND-white集合了女性化的柔美还有不变的街头风采，<UNK><UNK>的向日葵花朵美丽的出现在胸前和背后，犹如暗<UNK>闪光的星星一般耀眼又充满着<UNK>的生命力，而后品牌标志性的logo<UNK>出现，呈现出将花束固定的效果，有趣极了，穿的不仅是服饰。更是新颖创意的载体。"}

```

我们需要修改成单轮对话的数据微调格式。官方也提供了转换脚本，如下：

```
(chatglm3_multi) root@autodl-container-9052489e7e-Sac20027:/opt/muyu_chatglm3_6b/ChatGLM3/finetune_chatmodel_demo/scripts# python format_advertise_gen.py --path 'AdvertiseGen/train.json'
(chatglm3_multi) root@autodl-container-9052489e7e-Sac20027:/opt/muyu_chatglm3_6b/ChatGLM3/finetune_chatmodel_demo/scripts# ll
total 16784
drwxr-xr-x 4 root root 4096 Jan 10 12:57 ./
drwxr-xr-x 5 root root 4096 Jan 10 12:50 ../
drwxr-xr-x 2 root root 59 Jan 10 12:52 AdvertiseGen/
-rw-r--r-- 1 root root 1153 Jan 10 12:11 finetune_ds.sh
-rw-r--r-- 1 root root 1010 Jan 10 12:11 finetune_ds_multiturn.sh
-rw-r--r-- 1 root root 1124 Jan 10 12:11 finetune_pt.sh
-rw-r--r-- 1 root root 1048 Jan 10 12:11 finetune_pt_multiturn.sh
-rw-r--r-- 1 root root 585 Jan 10 12:11 format_advertise_gen.py
-rw-r--r-- 1 root root 1822 Jan 10 12:11 format_tool_alpaca.py
drwxr-xr-x 2 root root 59 Jan 10 12:52 formatted_data/
-rw-r--r-- 1 root root 17069994 Jan 10 12:56 index.html?dl=1
(chatglm3_multi) root@autodl-container-9052489e7e-Sac20027:/opt/muyu_chatglm3_6b/ChatGLM3/finetune_chatmodel_demo/scripts# cd formatted_data/
(chatglm3_multi) root@autodl-container-9052489e7e-Sac20027:/opt/muyu_chatglm3_6b/ChatGLM3/finetune_chatmodel_demo/scripts/formatted_data# ll
total 68024
drwxr-xr-x 2 root root 70 Jan 10 12:59 ./
drwxr-xr-x 4 root root 4096 Jan 10 12:57 ../
-rw-r--r-- 1 root root 53763260 Jan 10 12:59 advertise_gen.jsonl
-rw-r--r-- 1 root root 15886337 Jan 10 12:52 tool_alpaca.jsonl
(chatglm3_multi) root@autodl-container-9052489e7e-Sac20027:/opt/muyu_chatglm3_6b/ChatGLM3/finetune_chatmodel_demo/scripts/formatted_data#
```

执行后，数据格式如下：

```
{"prompt": "类型#裤*版型#宽松*风格#性感*图案#线条*裤型#阔腿裤",  
"response": "宽松的阔腿裤这两年真的吸粉不少，明星时尚达人的心头爱。毕竟好穿时尚，谁都能穿出腿长2米的效果宽松的裤腿，当然是遮肉小能手啊。上身随性自然不拘束，面料亲肤舒适贴身体验感棒棒哒。系带部分增加设计看点，还让单品的设计感更强。腿部线条若隐若现的，性感撩人。颜色敲温柔的，与裤子本身所呈现的风格有点反差萌。"}}
```

```
{"prompt": "类型#裙*风格#简约*图案#条纹*图案#线条*图案#撞色*裙型#鱼尾裙*裙袖长#无袖",  
"response": "圆形领口修饰脖颈线条，适合各种脸型，耐看有气质。无袖设计，尤显清凉，简约横条纹装饰，使得整身人鱼造型更为生动立体。加之撞色的鱼尾下摆，深邃富有诗意。收腰包臀，修饰女性身体曲线，结合别出心裁的鱼尾裙摆设计，勾勒出自然流畅的身体轮廓，展现了婀娜多姿的迷人姿态。"}}
```

单轮微调官方提供的示例脚本是 `finetune_pt.sh`。

```
(chatglm3 multi) root@autodl-container-fff445a19e-a56aeebe:~/autodl-tmp/muyu_chatglm3/ChatGLM3/finetune_chatmodel_demo# cd scripts/  
(chatglm3 multi) root@autodl-container-fff445a19e-a56aeebe:~/autodl-tmp/muyu_chatglm3/ChatGLM3/finetune_chatmodel_demo/scripts# ll  
total 32  
drwxr-xr-x 3 root root 4096 Jan 10 16:48 ./  
drwxr-xr-x 6 root root 4096 Jan 10 16:56 ../  
-rw-r--r-- 1 root root 1153 Jan 10 16:39 finetune_ds.sh  
-rw-r--r-- 1 root root 1010 Jan 10 16:39 finetune_ds_multiturn.sh  
-rw-r--r-- 1 root root 1177 Jan 10 16:46 finetune_pt.sh  
-rw-r--r-- 1 root root 1048 Jan 10 16:39 finetune_pt_multiturn.sh  
-rw-r--r-- 1 root root 585 Jan 10 16:39 format_advertise_gen.py  
-rw-r--r-- 1 root root 1822 Jan 10 16:39 format_tool_alpaca.py  
drwxr-xr-x 2 root root 33 Jan 10 16:43 formatted_data/
```

单轮对话使用P-Turning v2微调

准备完成后，需要安装一下执行微调过程必要的依赖包。执行如下命令：

```
pip install -r ../requirements.txt -i https://pypi.tuna.tsinghua.edu.cn/simple
```

```
(chatglm3 multi) root@autodl-container-9052489e7e-5ac20027:/opt/muyu_chatglm3_6b/ChatGLM3/finetune_chatmodel_demo# cat requirements.txt  
transformers==4.36.2  
datasets==2.16.0  
astunparse==1.6.3  
accelerate==0.25.0  
sentencepiece==0.1.99  
deepspeed==0.12.6  
(chatglm3 multi) root@autodl-container-9052489e7e-5ac20027:/opt/muyu_chatglm3_6b/ChatGLM3/finetune_chatmodel_demo#  
(chatglm3 multi) root@autodl-container-9052489e7e-5ac20027:/opt/muyu_chatglm3_6b/ChatGLM3/finetune_chatmodel_demo# pip install -r requirements.txt  
Looking in indexes: http://mirrors.aliyun.com/pypi/simple  
Requirement already satisfied: transformers==4.36.2 in /root/miniconda3/envs/chatglm3_multi/lib/python3.11/site-packages (from -r requirements.txt (line 1)) (4.36.2)  
Collecting datasets==2.16.0 (from -r requirements.txt (line 2))  
Using cached http://mirrors.aliyun.com/pypi/packages/ec/93/454ada0d1b289a0f4a86ac88dbdeab54921becabac45da3da787d136628f/datasets-2.16.1-py3-none-any.whl (507 kB)  
Collecting astunparse==1.6.3 (from -r requirements.txt (line 3))  
Using cached http://mirrors.aliyun.com/pypi/packages/2b/03/134de5512ad7b4557eb792fbc0c653af6076b81e5941d36ec61f7ce6028/astunparse-1.6.3-py2.py3-none-any.whl (12 kB)  
Requirement already satisfied: accelerate==0.25.0 in /root/miniconda3/envs/chatglm3_multi/lib/python3.11/site-packages (from -r requirements.txt (line 4)) (0.25.0)  
Requirement already satisfied: sentencepiece==0.1.99 in /root/miniconda3/envs/chatglm3_multi/lib/python3.11/site-packages (from -r requirements.txt (line 5)) (0.1.99)  
Collecting deepspeed==0.12.6 (from -r requirements.txt (line 6))  
Downloading http://mirrors.aliyun.com/pypi/packages/f1/ff/0fba0fec99e7de1c7148b0527e8ac9cdf2280d274ed135cb2187f7497a7/deepspeed-0.12.6.tar.gz (1.2 MB)  
Installing collected packages: deepspeed, datasets, astunparse  
Successfully installed astunparse-1.6.3 datasets-2.16.0 deepspeed-0.12.6
```

安装这些依赖

这里我们先使用默认的参数，仅修改必要的参数，把微调启动起来。

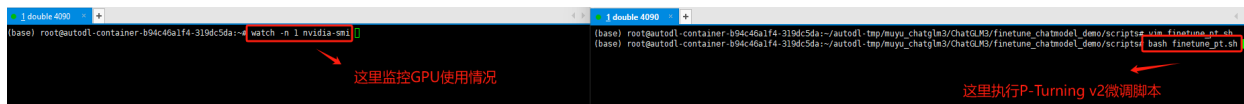
```
#!/usr/bin/env bash  
  
set -ex  
  
export NCCL_P2P_DISABLE=1  
export NCCL_IB_DISABLE=1  
  
PRE_SEQ_LEN=128  
LR=2e-2  
NUM_GPUS=1  
MAX_SOURCE_LEN=1024  
MAX_TARGET_LEN=128  
DEV_BATCH_SIZE=1  
GRAD_ACCUMULATION_STEPS=32  
MAX_STEP=1000  
SAVE_INTERVAL=500  
  
AUTORESUME_FROM_CHECKPOINT=True  
DATESTR="date +%Y%m%d-%H%M%S"  
RUN_NAME=advertise_gen_pt  
  
BASE_MODEL_PATH=../chatglm3-6b  
DATASET_PATH=formatted_data/advertise_gen.jsonl  
OUTPUT_DIR=output/${RUN_NAME}-${DATESTR}-${PRE_SEQ_LEN}-${LR}  
  
mkdir -p $OUTPUT_DIR  
  
torchrun --standalone --nnodes=1 --nproc_per_node=$NUM_GPUS ../finetune.py  
--train_format input-output \  
--train_file $DATASET_PATH \  
--preprocessing_num_workers 1 \  
--model_name_or_path $BASE_MODEL_PATH \  
--output_dir $OUTPUT_DIR \  
--max_source_length $MAX_SOURCE_LEN \  
--max_target_length $MAX_TARGET_LEN \  
--per_device_train_batch_size $DEV_BATCH_SIZE \  
--
```

如果使用的是消费级显卡，如3090或4090，添加这两个环境变量，因为消费级显卡不支持P2P（点对点）和IB（infinBand）这种更快的通信带宽

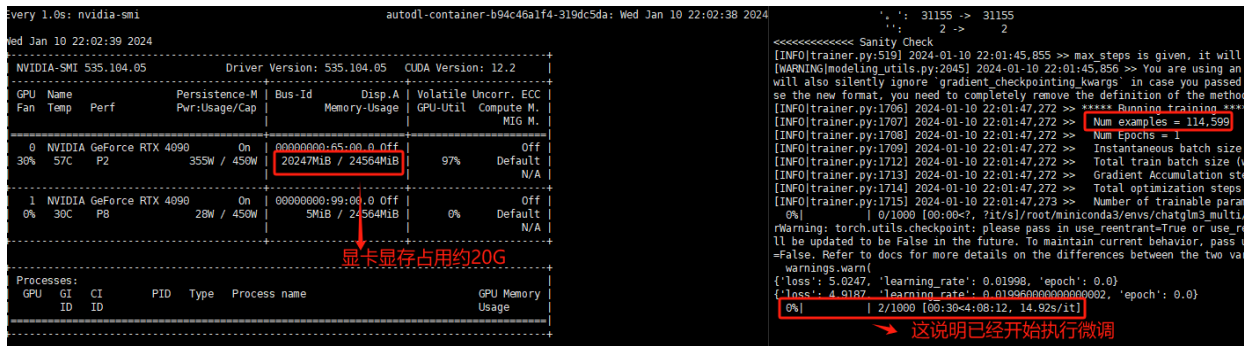
修改为本地chatglm3-6b模型的实际存储路径

修改执行微调主文件的.py路径

执行微调过程，并启动GPU显存使用情况的监控。



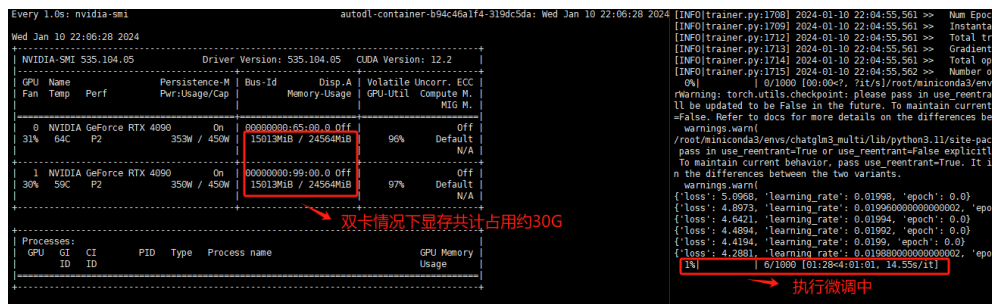
默认是使用单卡启动，会占用20GB的显存资源。



如果需要加载多块卡，可以进入 finetune_pt.sh 中修改一下配置。



双卡启动会占用30G的显存，占用更多显存的原因，主要会涉及一些模型复制、数据并行、梯度同步等训练过程中的操作。



参数名	描述
PRE_SEQ_LEN	Prompts序列的长度。
LR	学习率。高的学习率模型权重在优化过程中更新得更快。
NUM_GPUS	训练过程中使用的 GPU 数量。
MAX_SOURCE_LEN	定义输入序列的最大长度。
MAX_TARGET_LEN	定义输出序列的最大长度。
DEV_BATCH_SIZE	每个批次的大小，即每个优化步骤使用的示例数量。
GRAD_ACCUMULATION_STEPS	在进行一次参数更新之前，梯度积累的步数。模型会在 指定次数前向和后向传播后才更新参数。
MAX_STEP	训练过程执行的最大步数，
SAVE_INTERVAL	每隔一定数量的步数保存模型的参数

参数名	描述
RUN_NAME	运行的名称 (advertise_gen_pt)，用于标识和区分不同的训练运行。
BASE_MODEL_PATH	预训练模型的路径，保持为
DATASET_PATH	训练数据集的路径，保持为
OUTPUT_DIR	保存模型输出以及训练日志的文件夹路径。

因为默认参数是1000步，会导致训练过程较慢，这里我出于演示目的，将其调整为50执行微调。

```
#!/usr/bin/env bash

set -ex

export NCCL_P2P_DISABLE=1
export NCCL_IB_DISABLE=1

PRE_SEQ_LEN=128
LR=2e-2
NUM_GPUS=2
MAX_SOURCE_LEN=1024
MAX_TARGET_LEN=128
DEV_BATCH_SIZE=1
GRAD_ACCUMULATION_STEPS=32
MAX_STEP=50
SAVE_INTERVAL=500

AUTORESUME_FROM_CHECKPOINT=True
DATESTR=`date +%Y%m%d-%H%M%S`
RUN_NAME=advertise_gen_pt
```

如果使用的是消费级显卡，如3090 或 4090 添加这两个环境变量，因为消费级显卡不支持 P2P (点对点) 和IB (infiniBand) 这种更快的通信带宽

微调完成后，如下：

```
100%|██████████| 50/50 [12:06<00:00, 14.54s/it]
Saving PrefixEncoder
[INFO]configuration_utils.py:483] 2024-01-10 17:16:21,268 >> Configuration saved in output/advertise_gen_pt-20240110-170356-128-2e-2/config.json
[INFO]configuration_utils.py:594] 2024-01-10 17:16:21,269 >> Configuration saved in output/advertise_gen_pt-20240110-170356-128-2e-2/generation_config.json
[INFO]modeling_utils.py:2382] 2024-01-10 17:16:21,282 >> Model weights saved in output/advertise_gen_pt-20240110-170356-128-2e-2/pytorch_model.bin
[INFO]tokenization_utils_base.py:2432] 2024-01-10 17:16:21,283 >> tokenizer config file saved in output/advertise_gen_pt-20240110-170356-128-2e-2/tokenizer_config.json
[INFO]tokenization_utils_base.py:2441] 2024-01-10 17:16:21,283 >> Special tokens file saved in output/advertise_gen_pt-20240110-170356-128-2e-2/special_tokens_map.json
```

同时，会在 output/ 路径下会生成对应的模型文件：

```
drwxr-xr-x 4 root root 4096 Jan 10 17:03 ./
drwxr-xr-x 6 root root 4096 Jan 10 16:56 ../
-rw-r--r-- 1 root root 1153 Jan 10 16:39 finetune_ds.sh
-rw-r--r-- 1 root root 1010 Jan 10 16:39 finetune_ds_multiturn.sh
-rw-r--r-- 1 root root 1177 Jan 10 17:01 finetune_pt.sh
-rw-r--r-- 1 root root 1048 Jan 10 16:39 finetune_pt_multiturn.sh
-rw-r--r-- 1 root root 585 Jan 10 16:39 format_advertise_gen.py
-rw-r--r-- 1 root root 1822 Jan 10 16:39 format_tool_alpaca.py
drwxr-xr-x 2 root root 33 Jan 10 16:43 formatted_data/
drwxr-xr-x 3 root root 55 Jan 10 17:03 output/
```



```
drwxr-xr-x 2 root root 4096 Jan 10 17:16 advertise_gen_pt-20240110-170356-128-2e-2/
drwxr-xr-x 3 root root 4096 Jan 10 17:54 advertise_gen_pt-20240110-174150-128-2e-2/
(chatglm3_multi) root@autodl-container-fff445a19e-a56aeebe:~/autodl-tmp/muyu_chatglm3/ChatGLM3/finetune_chatmodel_demo# cd scripts/output/advertise_gen_pt-20240110-174150-128-2e-2/
(chatglm3_multi) root@autodl-container-fff445a19e-a56aeebe:~/autodl-tmp/muyu_chatglm3/ChatGLM3/finetune_chatmodel_demo# ls
scripts/output/advertise_gen_pt-20240110-174150-128-2e-2# ll
total 8332
drwxr-xr-x 3 root root 4096 Jan 10 17:54 ./
drwxr-xr-x 4 root root 116 Jan 10 17:41 ../
-rw-r--r-- 1 root root 4096 Jan 10 17:54 checkpoint-50/
-rw-r--r-- 1 root root 1468 Jan 10 17:54 config.json
-rw-r--r-- 1 root root 2332 Jan 10 17:54 configuration_chatglm.py
```

checkpoint 中存储的是训练过程中保存的模型状态，包括模型参数、优化器状态、当前epoch等信息。

```
-rw-r--r-- 1 root root 1468 Jan 10 17:54 config.json
-rw-r--r-- 1 root root 2332 Jan 10 17:54 configuration_chatglm.py
-rw-r--r-- 1 root root 133 Jan 10 17:54 generation_config.json
-rw-r--r-- 1 root root 55678 Jan 10 17:54 modeling_chatglm.py
-rw-r--r-- 1 root root 14682210 Jan 10 17:54 optimizer.pt
-rw-r--r-- 1 root root 7341306 Jan 10 17:54 pytorch_model.bin
-rw-r--r-- 1 root root 14692 Jan 10 17:54 quantization.py
-rw-r--r-- 1 root root 14512 Jan 10 17:54 rng_state_0.pth
-rw-r--r-- 1 root root 14512 Jan 10 17:54 rng_state_1.pth
-rw-r--r-- 1 root root 1064 Jan 10 17:54 scheduler.pt
-rw-r--r-- 1 root root 3 Jan 10 17:54 special_tokens_map.json
-rw-r--r-- 1 root root 12977 Jan 10 17:54 tokenization_chatglm.py
-rw-r--r-- 1 root root 1018370 Jan 10 17:54 tokenizer.model
-rw-r--r-- 1 root root 700 Jan 10 17:54 tokenizer_config.json
-rw-r--r-- 1 root root 5822 Jan 10 17:54 trainer_state.json
-rw-r--r-- 1 root root 4984 Jan 10 17:54 training_args.bin
```

不同的训练参数，会产生不同数量的 checkpoint，比如在脚本中，SAVE_INTERVAL 设置为1000，这说明每1000个训练步骤保存一次模型。如果 MAX_STEP 设置为3000，就应该有3个checkpoints被保存，这个也很好计算。

• 微调完成后使用微调模型执行推理

对于输入输出格式的微调，可以使用 inference.py 进行基本的推理验证。在 finetune_chatmodel_demo 文件目录中输入如下命令：

```
python inference.py --tokenizer 'chatglm3-6b模型路径' -- model '微调模型的checkpoint路径'
```

```
(chatglm3_multi) root@autodl-container-fff445a19e-a56aeebe:~/autodl-tmp/muyu_chatglm3/ChatGLM3/finetune_chatmodel_demo# python inference.py --pt-checkpoint 'scripts/output/advertise_gen_pt-20240110-174150-128-2e-2/checkpoint-50/' --model './chatglm3-6b/'
Loading checkpoint shards: 100%|#####| 7/7 [00:01<00:00, 4.38it/s]
Some weights of ChatGLMForConditionalGeneration were not initialized from the model checkpoint at ./chatglm3-6b/ and are newly initialized: ['transformer.prefix_encoder.embedding.weight']
You should probably TRAIN this model on a down-stream task to be able to use it for predictions and inference.
Prompt: 类型#裤#版型#宽松#风格#性感#图案#线条#裤型#阔腿裤
Response: 这条宽松的阔腿裤版型非常修饰身形，侧边线条的线条感非常明显，侧边有收腰设计，可以很好的修饰身形，侧边线条的线条感非常明显，侧边有收腰设计，可以很好的修饰身形。裤型宽松，可以很好的修饰身形，侧边线条的线条感非常明显，侧边有收腰设计，可以很好的修饰身形。
Prompt:
```

这是因为在 P-tuning v2 训练时模型只保存 PrefixEncoder 部分的参数，所以在推理时需要同时加载原 ChatGLM-6B 模型以及 PrefixEncoder 的权重。

• 多轮对话微调

多轮对话微调示例采用 ChatGLM3 对话格式约定，基本上大多数使用的也都是多轮对话的方式。

```
[
{
  "conversations": [
    {
```

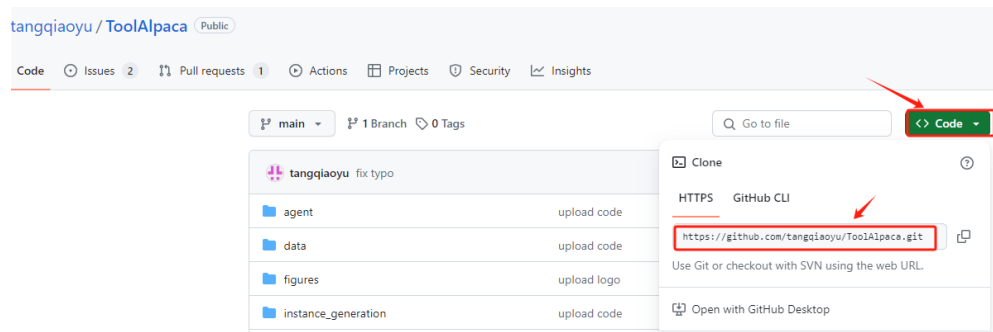


```

    "role": "system",
    "content": "<system prompt text>"
  },
  {
    "role": "user",
    "content": "<user prompt text>"
  },
  {
    "role": "assistant",
    "content": "<assistant response text>"
  },
  // ... Muti Turn
  {
    "role": "user",
    "content": "<user prompt text>"
  },
  {
    "role": "assistant",
    "content": "<assistant response text>"
  }
]
}
// ...
]

```

同样官方也提供了一个数据集，供用户快速使用，可以直接在github 上进行下载。



复制远程仓库的url链接后，直接在服务器上使用git工具拉取到本地。

```
(chatglm3_multi) root@autodl-container-fff445a19e-a56aeebe:~/autodl-tmp/muyu_chatglm3/ChatGLM3/fine
git clone https://github.com/tangqiaoyu/ToolAlpaca.git
Cloning into 'ToolAlpaca'...
remote: Enumerating objects: 79, done.
remote: Counting objects: 100% (79/79), done.
remote: Compressing objects: 100% (61/61), done.
remote: Total 79 (delta 18), reused 75 (delta 16), pack-reused 0
Unpacking objects: 100% (79/79), 3.58 MiB | 2.39 MiB/s, done.
(chatglm3_multi) root@autodl-container-fff445a19e-a56aeebe:~/autodl-tmp/muyu_chatglm3/ChatGLM3/fine
ll
total 64
drwxr-xr-x 7 root root 4096 Jan 10 18:08 ./
drwxr-xr-x 15 root root 4096 Jan 10 16:56 ../
drwxrwxr-x 2 1004 1004 52 Mar 18 2023 AdvertiseGen/
-rw-r--r-- 1 root root 8753 Jan 10 16:39 README.md
drwxr-xr-x 9 root root 4096 Jan 10 18:08 ToolAlpaca/
drwxr-xr-x 2 root root 126 Jan 10 16:45 pycache/
-rw-r--r-- 1 root root 4997 Jan 10 16:39 arguments.py
drwxr-xr-x 2 root root 36 Jan 10 16:39 configs/
-rw-r--r-- 1 root root 7003 Jan 10 16:39 finetune.py
-rw-r--r-- 1 root root 1986 Jan 10 16:39 inference.py
-rw-r--r-- 1 root root 6252 Jan 10 16:39 preprocess_utils.py
-rw-r--r-- 1 root root 114 Jan 10 16:39 requirements.txt
drwxr-xr-x 4 root root 4096 Jan 10 17:41 scripts/
-rw-r--r-- 1 root root 3279 Jan 10 16:39 trainer.py
(chatglm3_multi) root@autodl-container-fff445a19e-a56aeebe:~/autodl-tmp/muyu_chatglm3/ChatGLM3/fine
ll
```

多轮对话的微调数据集

同样，使用官方提供的格式转换脚本，转化成适合多轮微调格式的数据集。

```
(chatglm3_multi) root@autodl-container-fff445a19e-a56aeebe:~/autodl-tmp/muyu_chatglm3/ChatGLM
cd scripts/
(chatglm3_multi) root@autodl-container-fff445a19e-a56aeebe:~/autodl-tmp/muyu_chatglm3/ChatGLM
scripts# ll
total 32
drwxr-xr-x 4 root root 4096 Jan 10 17:41 ./
drwxr-xr-x 7 root root 4096 Jan 10 18:08 ../
-rw-r--r-- 1 root root 1153 Jan 10 16:39 finetune_ds.sh
-rw-r--r-- 1 root root 1010 Jan 10 16:39 finetune_ds_multiturn.sh
-rw-r--r-- 1 root root 1176 Jan 10 17:41 finetune_pt.sh
-rw-r--r-- 1 root root 1048 Jan 10 16:39 finetune_pt_multiturn.sh
-rw-r--r-- 1 root root 585 Jan 10 16:39 format_advertise_gen.py
-rw-r--r-- 1 root root 1822 Jan 10 16:39 format_tool_alpaca.py
drwxr-xr-x 2 root root 33 Jan 10 16:43 formatted_data/
drwxr-xr-x 4 root root 116 Jan 10 17:41 output/
(chatglm3_multi) root@autodl-container-fff445a19e-a56aeebe:~/autodl-tmp/muyu_chatglm3/ChatGLM
scripts# python format_tool_alpaca.py --path '../ToolAlpaca/data/train_data.json'
err_count: 207
train_examples: 4048
conversation distribution: Counter({4: 2513, 6: 746, 8: 382, 10: 176, 12: 100, 14: 52, 16: 20
28: 5, 26: 5, 24: 4, 30: 2})
(chatglm3_multi) root@autodl-container-fff445a19e-a56aeebe:~/autodl-tmp/muyu_chatglm3/ChatGLM
scripts# cd formatted_data/
(chatglm3_multi) root@autodl-container-fff445a19e-a56aeebe:~/autodl-tmp/muyu_chatglm3/ChatGLM
scripts/formatted_data# ll
total 68024
drwxr-xr-x 2 root root 58 Jan 10 18:09 ./
drwxr-xr-x 4 root root 4096 Jan 10 17:41 ../
-rw-r--r-- 1 root root 53763280 Jan 10 16:43 advertise_gen.jsonl
-rw-r--r-- 1 root root 15886337 Jan 10 18:09 tool_alpaca.jsonl
(chatglm3_multi) root@autodl-container-fff445a19e-a56aeebe:~/autodl-tmp/muyu_chatglm3/ChatGLM
scripts/formatted_data# ll
```

格式化数据集

这次使用 finetune_pt_multiturn.sh 微调脚本，进行和单轮对话微调相同的配置修改即可。

```
#!/usr/bin/env bash

set -ex

export NCCL_P2P_DISABLE=1
export NCCL_IB_DISABLE=1

PRE_SEQ_LEN=128
LR=2e-2
NUM_GPUS=1
MAX_SEQ_LEN=2048
DEV_BATCH_SIZE=1
GRAD_ACCUMULATION_STEPS=16
MAX_STEP=1000
SAVE_INTERVAL=500

AUTORESUME_FROM_CHECKPOINT=True
DATESTR="date +%Y%m%d-%H%M%S"
RUN_NAME=tool_alpaca_pt

BASE_MODEL_PATH=../chatglm3-6b
DATASET_PATH=formatted_data/tool_alpaca.jsonl
OUTPUT_DIR=output/${RUN_NAME}-${DATESTR}-${PRE_SEQ_LEN}-${LR}

mkdir -p $OUTPUT_DIR

torchrun --standalone --nnodes=1 --nproc_per_node=$NUM_GPUS ../finetune.py \
--train_format multi-turn \
--train_file $DATASET_PATH \
--max_seq_length $MAX_SEQ_LEN \
--preprocessing_num_workers 1 \
--model_name_or_path $BASE_MODEL_PATH \
--output_dir $OUTPUT_DIR \
--per_device_train_batch_size $DEV_BATCH_SIZE \
--gradient_accumulation_steps $GRAD_ACCUMULATION_STEPS \
--max_steps $MAX_STEP \
--logging_steps 1 \
--save_steps $SAVE_INTERVAL \
--learning_rate $LR \
--pre_seq_len $PRE_SEQ_LEN \
--resume_from_checkpoint $AUTORESUME_FROM_CHECKPOINT >&| tee ${OUTPUT_DIR}/train.log
```

修改为本地chatglm3-6b模型的实际存储路径

修改主文件路径

和单轮对话格式微调的区别在这里

微调过程中会占用24G显存。

```
Every 1.0s: nvidia-smi
```

GPU	Name	Temp	Perf	Persistence-M	Usage/Cap	Bus-Id	Memory-Usage	Disp.A	Volatile Uncorr. ECC	GPU-Util	Compute M.	MIG M.
0	NVIDIA GeForce RTX 4090	30C	53C	P2	363M / 450M	00000000:05:00:0	Off	23877MiB / 24564MiB	100%	Default	N/A	
1	NVIDIA GeForce RTX 4090	0%	31C	P8	27M / 450M	00000000:09:00:0	Off	5MiB / 24564MiB	0%	Default	N/A	

单卡显存同样会占用约24G

```
Warning: torch.utils.checkpoint: please pass in use_reentrant=True or
False. Refer to docs for more details on the differences between the t
warnings.warn(
  [0%] | 0/1000 [00:00<?, ?it/s] /root/miniconda3/envs/chatglm3
[0%] | 1/1000 [00:13<3:52:35, 13.97e/it]
```

执行到这说明微调已正常运行

其推理验证过程，和上面说明的单轮对话微调模型的一致。

训练过程的参数会很大程度影响当前训练的显存占用，比如我们做如下实验：

```
#!/usr/bin/env bash

set -ex

export NCCL_P2P_DISABLE=1
export NCCL_IB_DISABLE=1

PRE_SEQ_LEN=128
LR=2e-2
NUM_GPUS=2
MAX_SOURCE_LEN=1024
MAX_TARGET_LEN=128
DEV_BATCH_SIZE=16
GRAD_ACCUMULATION_STEPS=32
MAX_STEP=1000
SAVE_INTERVAL=500
```

这里改为16

显存直接就会爆掉：

```
torch.cuda.OutOfMemoryError: CUDA out of memory. Tried to allocate 4.47 GiB. GPU 1 has a total capacity of 23.65 GiB of which 3.06 GiB is free.
Process 105294 has 20.58 GiB memory in use. Of the allocated memory 18.95 GiB is allocated by PyTorch, and 1.09 GiB is reserved by PyTorch
h but unallocated. If reserved but unallocated memory is large try setting max_split_size_mb to avoid fragmentation. See documentation for
Memory Management and PYTORCH_CUDA_ALLOC_CONF
[0%] | 0/1000 [00:03<?, ?it/s]
[2024-01-10 22:14:11.657] torch.distributed.elastic.multiprocessing.api: [ERROR] failed (exitcode: 1) local_rank: 0 (pid: 6555) of binary: /
root/miniconda3/envs/chatglm3_multi/bin/python
Traceback (most recent call last):
  File "/root/miniconda3/envs/chatglm3_multi/bin/torchrun", line 8, in <module>
    sys.exit(main())
```

4090 双卡内存会直接爆掉

对于双卡 4090 共计48显存来说，最大仅支持设置到4,就基本会占满显存。

Every 1.0s: nvidia-smi									
Wed Jan 10 22:17:09 2024									
NVIDIA-SMI 535.104.05 Driver Version: 535.104.05 CUDA Version: 12.2									
GPU	Name	Perf.	Persistence-M	Bus-Id	Disp.A	Volatile Uncorr. ECC	GPU-Util	Compute M.	Memory Usage
Fan	Temp	Perf	Par:Usage/Cap						
0	NVIDIA GeForce RTX 4090	30%	On	00000000:65:00:0	Off	Off	100%	Default	22001MiB / 24564MiB
1	NVIDIA GeForce RTX 4090	30%	On	00000000:65:00:0	Off	Off	100%	Default	22001MiB / 24564MiB
Processes:									
GPU	CI	CI	PID	Type	Process name				GPU Memory Usage
ID	ID	ID							

如果按照这种 - per_device_train_batch_size=1、- gradient_accumulation_steps=16 比较低的参数设置还爆显存的话，只能尝试微调量化模型。

```
BASE_MODEL_PATH=../../chatglm3-6b
DATASET_PATH=formattd_data/advertise_gen.jsonl
OUTPUT_DIR=output/${RUN_NAME}-${DATESTR}-${PRE_SEQ_LEN}-${LR}

mkdir -p $OUTPUT_DIR

torchrun --standalone --nnodes=1 --nproc_per_node=$NUM_GPUS ../finetune.py \
--quantization bit 4 \
--train format input-output \
--train_file $DATASET_PATH \
```

INT4 的模型参数被冻结，一次训练迭代会以 1 的批处理大小进行 16 次累加的前后向传播，等效为 16 的总批处理大小，实际显存占用也仅有7.9G。

Every 1.0s: nvidia-smi									
Wed Jan 10 22:27:37 2024									
NVIDIA-SMI 535.104.05 Driver Version: 535.104.05 CUDA Version: 12.2									
GPU	Name	Perf.	Persistence-M	Bus-Id	Disp.A	Volatile Uncorr. ECC	GPU-Util	Compute M.	Memory Usage
Fan	Temp	Perf	Par:Usage/Cap						
0	NVIDIA GeForce RTX 4090	30%	On	00000000:65:00:0	Off	Off	100%	Default	7933MiB / 24564MiB
1	NVIDIA GeForce RTX 4090	0%	On	00000000:65:00:0	Off	Off	0%	Default	5MiB / 24564MiB
Processes:									
GPU	CI	CI	PID	Type	Process name				GPU Memory Usage
ID	ID	ID							

所以，P-Tuning V2 高效微调中，PRE_SEQ_LEN=128，DEV_BATCH_SIZE=1，GRAD_ACCUMULARION_STEPS=16，MAX_SEQ_LEN=2048 配置下约需要 21GB 显存。

若尝试后发现显存不足，可以考虑：

- 尝试降低 DEV_BATCH_SIZE 并提升 GRAD_ACCUMULARION_STEPS
- 尝试添加 --quantization_bit 8 或 --quantization_bit 4。

除此之外，对于模型参数的选择，往往是参数越大效果越好。如果资源充足，当然是推荐 30B 以上的模型。不管是 6B, 7B 和 13B 同样的训练数据，同样训练参数，模型参数量大效果则优于低参数的模型。根据模型参数预估训练所需的内存开销，一个简单的方法是：比如 6B 模型，60亿规模参数，根据以下公式计算：

模型参数 + 梯度参数 + 优化器参数 = 6B * 1bytes + 6GB + 2*6GB = 24GB

注意：参数多量化低的模型要优于参数低量化高的模型，举例：33B-fb4 模型要优于 13b-fb16 模型。

- 全量微调

对于全量微调，需要使用 finetune_ds_multiturn.sh 这个脚本，其配置 MAX_SEQ_LEN=2048，DEV_BATCH_SIZE=16，GRAD_ACCUMULATION_STEPS=1 恰好用满 4 * 80GB 显存。

硬件要求	ChatGLM-6B
推理	任意8GB显存以上GPU * 1
微调	A100 (80G) * 4 A100 (40G) * 8 V100 (32G) * 12

我这里也尝试了一下，可以跑通：

```
#!/usr/bin/env bash
set -ex
export NCCL_P2P_DISABLE=1
export NCCL_IB_DISABLE=1

NPROC=4
MAX_SEQ_LEN=2048
MAX_TARGET_LEN=128
DEV_BATCH_SIZE=16
GRAD_ACCUMULATION_STEPS=1
MAX_STEPS=1000000
SAVE_INTERVAL=1000

AUTORESUME_FROM_CHECKPOINT=True
RUN_NAME=advtest_qwen_ft
BASE_MODEL_PATH=../chatglm3-6b
DATASET_PATH=formatted_data/advtest_gen.jsonl

DATESTR=$(date +%Y%m%d-%H%M%S)
OUTPUT_DIR=output/${RUN_NAME}-${DATESTR}-${SLURM_JOB_ID}
MASTER_PORT=$(shuf -n 1 -i 10000-65535)

mkdir -p $OUTPUT_DIR

torchrun --standalone --nnodes=1 --nproc_per_node=$NPROC --master_port=$MASTER_PORT -- ./finetune.py \
--train_format input-output \
--train_file $DATASET_PATH \
--preprocessing_num_workers 1 \
--model_name_or_path $BASE_MODEL_PATH \
--output_dir $OUTPUT_DIR \
--max_source_length $MAX_SOURCE_LEN \
--max_target_length $MAX_TARGET_LEN \
--per_device_train_batch_size $DEV_BATCH_SIZE \
--gradient_accumulation_steps $GRAD_ACCUMULATION_STEPS \
--max_steps $MAX_STEPS \
--logging_steps 1 \
--save_steps $SAVE_INTERVAL \
--learning_rate SLR \
--fp16 \
--deepspeed ../configs/deepspeed.json \
--resume_from_checkpoint $AUTORESUME_FROM_CHECKPOINT >>> | tee $(OUTPUT_DIR)/train.log
```

但奈何硬件差距太多，直接显存爆掉。

```
torch.cuda.OutOfMemoryError: CUDA out of memory. Tried to allocate 11.63 GiB. GPU 0 has a total capacity of 23.65 GiB of which 5.72 GiB is fr
ee. Process 119420 has 17.92 GiB memory in use. Of the allocated memory 17.44 GiB is allocated by PyTorch, and 5.09 MiB is reserved by PyTorch
but unallocated. If reserved but unallocated memory is large try setting max_split_size_mb to avoid fragmentation. See documentation for
Memory Management and PYTORCH_CUDA_ALLOC_CONF
(2024-01-10 22:31:22.115) torch.distributed.elastic.multiprocessing.api: [ERROR] failed (exitcode: 1) local_rank: 0 (pid: 124355) of binary:
/root/miniconda3/envs/chatglm3_multi/bin/python
Traceback (most recent call last):
  File "/root/miniconda3/envs/chatglm3_multi/bin/torchrun", line 8, in <module>
    sys.exit(main())
  File "/root/miniconda3/envs/chatglm3_multi/lib/python3.11/site-packages/torch/distributed/elastic/multiprocessing/errors/_init_.py", line
346, in wrapper
```

4. 大模型并行训练框架-DeepSpeed

训练像ChatGLM3-6B这种大的模型往往需要配备高价的多GPU、多节点的集群，但是，即便拥有了这些先进的硬件资源，实际的机器利用率往往只能达到其最大效率的一半左右。这意味着，仅仅拥有更加强大的硬件资源并不能保证更高的模型训练吞吐量。同样，即使系统具有更高的吞吐量，也并不能保证所训练出的模型具有更高的精度或更快的收敛速度。更重要的是，当前的开源软件的易用性也常常被用户诟病。

DeepSpeed是一个开源深度学习优化库，专门设计来提高大型模型训练的效率 and 扩展性。这个库采用了一系列先进技术，如模型并行化、梯度累积、动态精度缩放和混合精度训练等，来实现快速训练。除此之外，DeepSpeed还搭载了一套强大的辅助工具集，涵盖分布式训练管理、内存优化以及模型压缩等功能，帮助开发者更有效地处理和优化大规模的深度学习任务。值得一提的是，DeepSpeed是基于PyTorch构建的，因此对于现有的PyTorch项目，开发者可以轻松地实施迁移。此库已在众多大规模深度学习应用中得到验证，涉及领域包括但不限于语言模型、图像分类和目标检测。

其使用非常简单，其较强的易用性源于把该软件的构造难度交给开发者而不是用户，所以我们用起来是非常简单的，就是一个configs文件，然后在训练代码中反向传播后执行参数更新的时候加一两行代码就可以了。对于ChatGLM3-6B模型的微调，默认只是在全量微调的脚本中加入了deepspeed的代码，因硬件配置相差太大，即使是使用deepspeed也无法运行。但我们可以将其应用到高效微调的P-Tuning v2中，只需要添加一行代码，其他的全部使用默认的即可。

DeepSpeed已经在Github上开源，地址：<https://github.com/microsoft/DeepSpeed>

这里在 `fineturn_pt.sh` 中加入这样一行代码：

```
#!/usr/bin/env bash

set -ex

export NCCL_P2P_DISABLE=1
export NCCL_IB_DISABLE=1

PRE_SEQ_LEN=128
LR=2e-2
NUM_GPUS=2
MAX_SOURCE_LEN=1024
MAX_TARGET_LEN=128
DEV_BATCH_SIZE=4
GRAD_ACCUMULATION_STEPS=16
MAX_STEP=1000
SAVE_INTERVAL=500

AUTORESUME_FROM_CHECKPOINT=True
DATESTR=`date +%Y%m%d-%H%M%S`
RUN_NAME=advertise_gen_pt

BASE_MODEL_PATH=../../chatglm3-6b
DATASET_PATH=formatted_data/advertise_gen.jsonl
OUTPUT_DIR=output/${RUN_NAME}-${DATESTR}-${PRE_SEQ_LEN}-${LR}

# MASTER_PORT=$(shuf -n 1 -i 10000-65535)

mkdir -p $OUTPUT_DIR

torchrun --standalone --nnodes=1 --nproc_per_node=$NUM_GPUS ../finetune.py \
  [-deepspeed ../configs/deepspeed.json \
  --train_format input-output \
  --train_file $DATASET_PATH \
  --preprocessing_num_workers 1 \
  --model_name_or_path $BASE_MODEL_PATH \
  --output_dir $OUTPUT_DIR \
  --max_source_length $MAX_SOURCE_LEN \
  --max_target_length $MAX_TARGET_LEN \
  --per_device_train_batch_size $DEV_BATCH_SIZE \
  --gradient_accumulation_steps $GRAD_ACCUMULATION_STEPS \
  --max_steps $MAX_STEP \
  --logging_steps 1 \
  --save_steps $SAVE_INTERVAL \
  --learning_rate $LR \
  --pre_seq_len $PRE_SEQ_LEN \
  --resume_from_checkpoint $AUTORESUME_FROM_CHECKPOINT 2>&1 | tee ${OUTPUT_DIR}/train.log
```

添加这一行启用DeepSpeed加速

直接启动后，就会降低约4G的显存使用。因我们的配置和数据量过小，其实不太容易看出这种差距。训练级别越高，提升的效果会越明显。


```
Every 1.0s: nvidia-smi                 autodl-container-b94c46a1f4-319dc5da: Wed Jan 10 23:00:27 2024
+-----+
| NVIDIA-SMI 535.104.05                Driver Version: 535.104.05   CUDA Version: 12.2   |
+-----+-----+-----+-----+-----+-----+
| GPU  Name      Persistence-M | Bus-Id  Disp.A | Volatile Uncorr. ECC |
| Fan  Temp  Perf    Pwr:Usage/Cap |         Memory-Usage | GPU-Util  Compute M. |
|-----+-----+-----+-----+-----+-----+
| 0  NVIDIA GeForce RTX 4090   On      | 00000000:00:00:00 Off |                     |
| 30%   53C   P2              265W / 450W | 20735MiB / 24564MiB | 100%      Default  |
+-----+-----+-----+-----+-----+-----+
| 1  NVIDIA GeForce RTX 4090   On      | 00000000:99:00:00 Off |                     |
| 30%   50C   P2              271W / 450W | 20735MiB / 24564MiB | 100%      Default  |
+-----+-----+-----+-----+-----+-----+
| Processes:                               GPU Memory |
|  GPU   GI   CI          PID    Type   Process name                  Usage |
|  -----+-----+-----+-----+-----+-----+
| 0      0    0         4997      R   python3                        28M |
| 1      0    0         7003      R   python3                        19M |
| 1      0    0         1986      R   python3                        62M |
| 1      0    0         6252      R   python3                        114M |
| 1      0    0         4096      R   python3                        4096M |
| 1      0    0         3279      R   python3                        490M |
+-----+-----+-----+-----+-----+-----+

[INFO]trainer.py:1706] 2024-01-10 22:59:46,611 >> ***** Running training *****
[INFO]trainer.py:1707] 2024-01-10 22:59:46,612 >> Num examples = 114,599
[INFO]trainer.py:1708] 2024-01-10 22:59:46,612 >> Num Epochs = 2
[INFO]trainer.py:1709] 2024-01-10 22:59:46,612 >> Instantaneous batch size per device = 4
[INFO]trainer.py:1712] 2024-01-10 22:59:46,612 >> Total train batch size (w. parallel, distributed) = 16
[INFO]trainer.py:1713] 2024-01-10 22:59:46,612 >> Gradient Accumulation steps = 16
[INFO]trainer.py:1714] 2024-01-10 22:59:46,612 >> Total optimization steps = 1,000
[INFO]trainer.py:1715] 2024-01-10 22:59:46,613 >> Number of trainable parameters = 1,835,008
[INFO]trainer.py:1716] 2024-01-10 22:59:46,613 >> 0/1000 [00:00<, 71it/s] /root/miniconda3/envs/chatglm3_multi/lib/python3.11/site-packages/torch/nn/modules/module.py:1131: UserWarning: torch.nn.modules.module: please pass in use_reentrant=True or use_reentrant=False explicitly. It will be updated to be False in the future. To maintain current behavior, pass use_reentrant=True. It is recommended that you use use_reentrant=False. Refer to docs for more details on the differences between the two variants.
  warnings.warn(
/root/miniconda3/envs/chatglm3_multi/lib/python3.11/site-packages/torch/nn/modules/module.py:1131: UserWarning: torch.nn.modules.module: please pass in use_reentrant=True or use_reentrant=False explicitly. The default value of use_reentrant will be updated to be False in the future. To maintain current behavior, pass use_reentrant=True. It is recommended that you use use_reentrant=False. Refer to docs for more details on the differences between the two variants.
  warnings.warn(
/root/miniconda3/envs/chatglm3_multi/lib/python3.11/site-packages/deepspeed/runtime/zero/stage_1_and_2.py:429: UserWarning: DTypeTensor constructors are no longer recommended. It's best to use methods such as torch.tensor(data, dtype=dtype, device=device, pin_memory=pin_memory) or torch.nn.Parameter(torch.nn.init._fill_(torch.nn.Parameter(torch.empty([1], dtype=dtype, device=device, pin_memory=pin_memory))), value) instead. (Triggered internally at ../torch/csrc/tensor/python_tensor.cpp:83.)
  total_norm_cuda = get_accelerator().FloatTensor([float(total_norm)])
/root/miniconda3/envs/chatglm3_multi/lib/python3.11/site-packages/deepspeed/runtime/zero/stage_1_and_2.py:429: UserWarning: DTypeTensor constructors are no longer recommended. It's best to use methods such as torch.tensor(data, dtype=dtype, device=device, pin_memory=pin_memory) or torch.nn.Parameter(torch.nn.init._fill_(torch.nn.Parameter(torch.empty([1], dtype=dtype, device=device, pin_memory=pin_memory))), value) instead. (Triggered internally at ../torch/csrc/tensor/python_tensor.cpp:83.)
  total_norm_cuda = get_accelerator().FloatTensor([float(total_norm)])
```

DeepSpeed降低约4GB显存占用

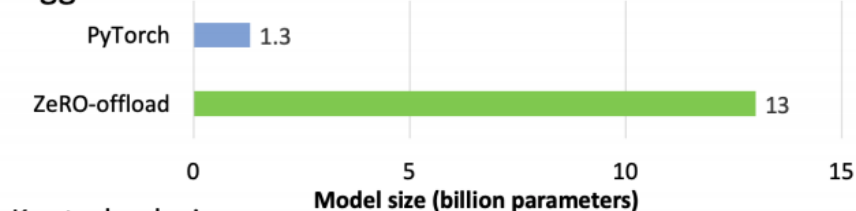
deepspeed在训练过程中依赖的相关参数配置，来源于这里：

```
-rw-r--r-- 1 root root 4997 Jan 10 16:39 arguments.py
drwxr-xr-x 2 root root 28 Jan 10 23:11 configs/
-rw-r--r-- 1 root root 7003 Jan 10 16:39 finetune.py
-rw-r--r-- 1 root root 1986 Jan 10 16:39 inference.py
-rw-r--r-- 1 root root 6252 Jan 10 16:39 preprocess_utils.py
-rw-r--r-- 1 root root 114 Jan 10 16:39 requirements.txt
drwxr-xr-x 4 root root 4096 Jan 10 23:11 scripts/
-rw-r--r-- 1 root root 3279 Jan 10 16:39 trainer.py
(base) root@autodl-container-b94c46a1f4-319dc5da:~/autodl-tmp/muyu_chatglm3/ChatGLM3
# ll
total 8
drwxr-xr-x 2 root root 28 Jan 10 23:11 ./
drwxr-xr-x 7 root root 4096 Jan 10 18:17 ../
-rw-r--r-- 1 root root 490 Jan 10 23:11 deepspeed.json
(base) root@autodl-container-b94c46a1f4-319dc5da:~/autodl-tmp/muyu_chatglm3/ChatGLM3
# cat deepspeed.json
{
  "train_micro_batch_size_per_gpu": "auto",
  "zero_allow_untested_optimizer": true,
  "fp16": {
    "enabled": "auto",
    "loss_scale": 0,
    "initial_scale_power": 16,
    "loss_scale_window": 1000,
    "hysteresis": 2,
    "min_loss_scale": 1
  },
  "zero_optimization": {
    "stage": 2,
    "allgather_partitions": true,
    "allgather_bucket_size": 5e8,
    "overlap_comm": false,
    "reduce_scatter": true,
    "reduce_bucket_size": 5e8,
    "contiguous_gradients": true
  }
}
```

这个参数的调整，直接影响整体的训练效率。但这部分参数需要对Deepspeed有一定的了解才能更好的根据训练任务和硬件配置情况灵活调整。我们这里简单的了解一下。

DeepSpeed是最早开始关注大模型训练的一批，其最核心的就是ZeRo，ZeRo-Offload是将模型的参数、梯度或者优化器的状态可以从GPU内存中转移到GPU中。

ZeRO-Offload (GPU + CPU): 13B model on single GPU, 10x bigger

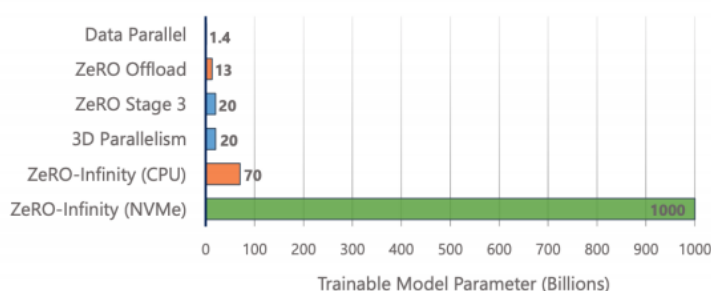


Key technologies:

- Heterogeneous memory, ZeRO-style data parallelism, efficient tensor allocation and migration

其终极形态ZeRo-infinity，不仅可以将这些参数等卸载到CPU上，还可以Offload到nvme的硬盘上，在速度上，基于zero的这种方式，随着GPU的增加，可以达到超线性的效率增长和。

ZeRO-Infinity (GPU + CPU + NVMe): 1 Trillion model on a single GPU, 700x bigger



第二点是训练速度。不管大模型还是小模型的训练，训练的效率一定是框架需要重点关注的，需要在保证精确性的前提下，保证它快。

Fastest Transformer Kernels



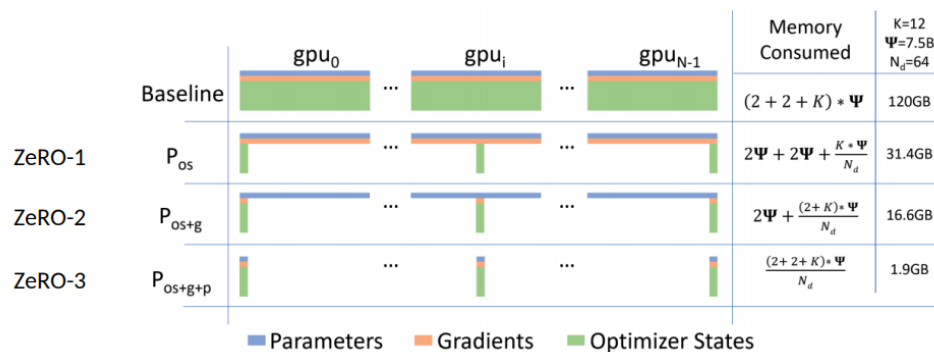
World Fastest BERT Training

#Devices	Source	Training Time
256 V100 GPUs	Nvidia	236 mins
256 V100 GPUs	DeepSpeed	144 mins
1024 TPU3 chips	Google	76 mins
1024 V100 GPUs	Nvidia	67 mins
1024 V100 GPUs	DeepSpeed	44 mins

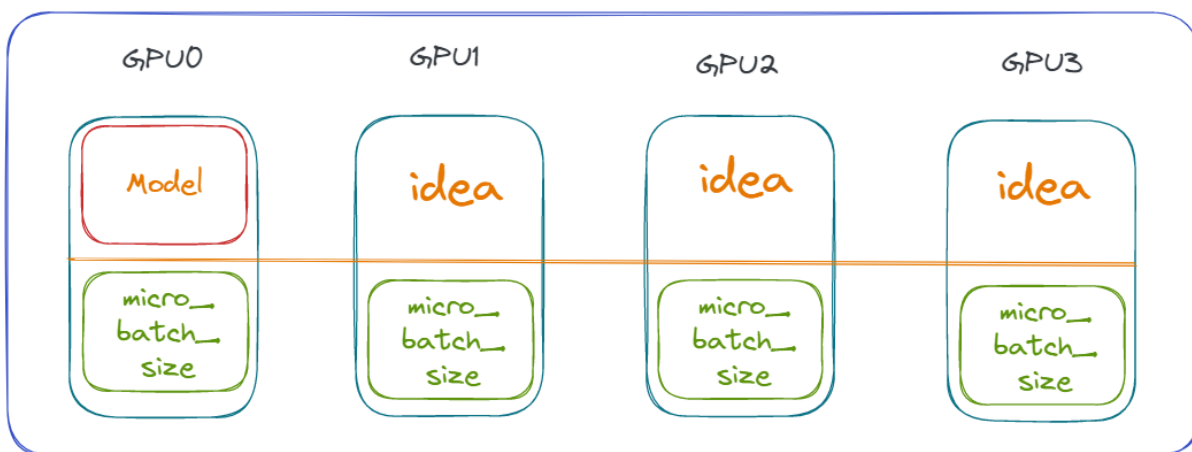
ZeRO-1只会对优化器状态做切分。ZeRO-2会对优化器状态和梯度做切分。ZeRO-3是对优化器状态、梯度和模型参数做切分。

- ϕ : 假设有一个模型，这个模型由 ϕ 个参数，也就是由 ϕ 个浮点数组成的模型，每个参数如果以fp16的形式存放，一个参数是32float，也就是4bit，所以这里就是 2ϕ 。
- 梯度，同样是 2ϕ 的显存占用。
- 优化器状态就是K倍的 ϕ ，优化器的状态根据实现的形式是不一样的，这里选择12进行比较。

在Baseline中，这120GB显存是每张卡都要占用的，所以现在最大的H100这种80G的显存都放不下。



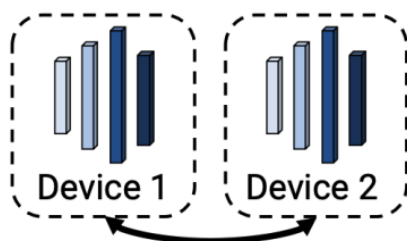
所以在实际计算过程中，GPU1 ~ GPU3 计算显存空间的使用会根据 GPU0 的可使用显存空间来确定，这就造成了一个问题：在显存使用上，GPU0 = GPU1 = GPU2 = GPU3，对 GPU1~GPU3 来说是一种巨大的浪费。而且，这种浪费随着模型精度、参数的增加愈发明显。



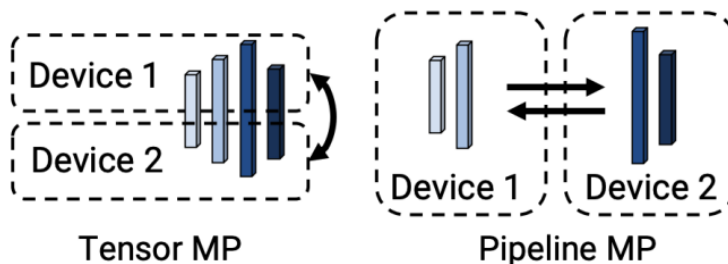
并行模型：

- Data Parallelism (DP)：数据并行，整个模型会复制到所有GPU上，输入数据会被分割成多个batch到不同的GPU上。也因为每个GPU都在处理不同的数据子集，所以在独立执行前向传播后计算的损失 (loss) 也会有所不同，接着每个GPU根据其计算出的损失执行反向传播，计算梯度。当所以GPU计算完成后，求平均。这个平均梯度代表了整个数据集上的平均梯度。使用这个平均梯度更新模型的参数。从而确保所有GPU上的模型都保持同步。
- Tensor MP：对模型做横向切分，也就是层内的切分，每一层的计算被分割成几个较小的部分，每部分独立在不同的GPU上进行计算。比如最大层是一个MLP层，有非常大的计算，但一张卡放不下，就需要切分成两个小的分别放在两张卡上计算。
- Pipeline MP：流水线并行，把模型的不同层分在不同的GPU上，比如12层的模型，前6层分在一个GPU上，后六层分在一个GPU上。像我们常用的Transformer结果，它会分成一个个Block，所以一般不同的Block会分布不同的层中。

Data Parallelism (DP)



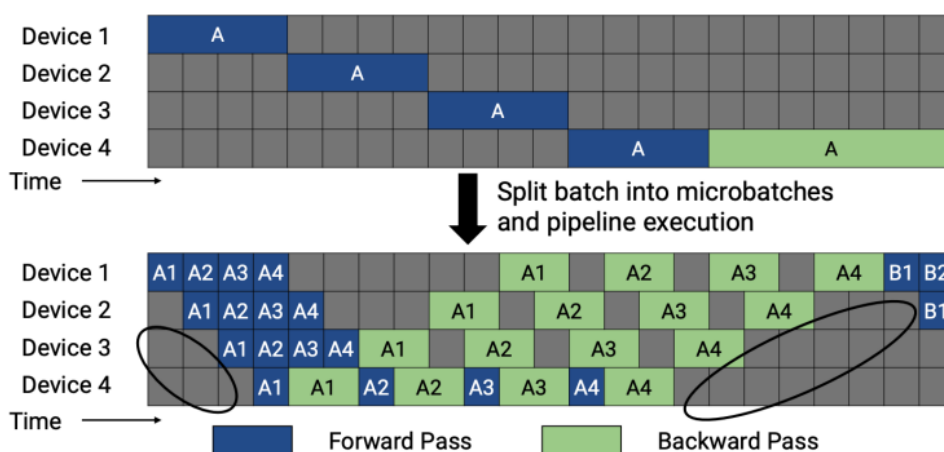
Model Parallelism (MP)



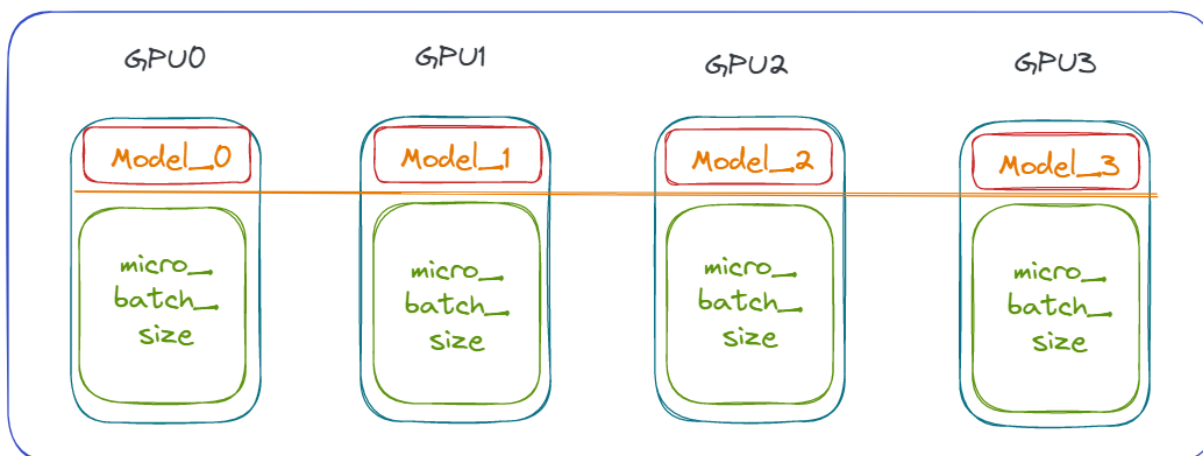
什么是micro_batch_size?

Pipeline会把输入进来的mini_batch 分成设备个 micro_batch。

Pipeline model parallelism



理想的计算加载方式应该是将模型加载在每个GPU上，减少模型对单个GPU的占用依赖，如下图所示：



DeepSpeed 就是实现这样的加载，结合Deepspeed框架的优化特性，充分发挥每块GPU的计算和显存潜能，从而提高整体的训练效率和资源利用率。

在理解了DeepSpeed原理后，我们尝试进行模型加载并观察其内存消耗情况。

对于我们的本地运行环境，如果采用 DeepSpeed 在4块3090上加载ChatGLM2-6B模型，加载情况如下：

- 原始模型大小 chatglm-6B FP16 -> 单卡显存 $Mem_{model}=12GB$
- 在 $n_{GPU} = 4$ 的情况下采用 zero++ 方式计算过程中, 模型会先加载到 内存中, 占用内存大小 $Men_{load} = n_{GPU} * Mem_{model} = 4 * 12 \approx 48GB$
- 内存加载完毕后再分布到各个显存上, 遵循 “对内存中 n_{GPU} 个模型进行截取, 而不是 一个模型进行分割”;
- 计算公式如下:
 $Men_{load}=Mem_{model} * n_{GPU}$
- 加载130B FP16 $n_{GPU} = 4$ 时, $Men_{load}=Mem_{model} * n_{GPU}=130*4=520GB$

可见, DeepSpeed 的 分布截取 会占用大量重复内存, 造成资源上的冲击和浪费, 一种优化方法是加载到虚拟内存中作为缓存, 对一个模型进行分割而不是逐个截取。

在微调过程中, 参数配置和优化对于模型性能和训练效率至关重要。合理的参数设置不仅可以加速模型的收敛, 还可以提高模型的表现。特别是当我们使用高级的训练框架如DeepSpeed时, 更需要对每个参数有深入的理解和精细的调整。DeepSpeed训练过程中涉及的主要参数和分类如下:

para	n	GPU	times/acc	cost	micro-bs	bs	1 0000	times
4-4-16	4U	21554MB	4	50s / step	64	256	39 step/epoch	32.5min
4-4-12	4U	21554MB	3	40s / step	48	192	52 step/epoch	34.6h
4-4-8	4U	21554MB	2	25s / step	32	128	78 step/epoch	32.5h
2-2-16	4U	18254MB	8	25s / step	32	128	78 step/epoch	32.5h
2-2-32	4U	18254MB	16	50s / step	64	256	39 step/epoch	32.5h

$micro-bs=TRAIN_BATCH_SIZE*GRA_ACC_STEPS$

$bs=micro-bs/n_{GPU}$

其中, para列中的4-4-16表示per_device_train_batch_size、per_device_eval_batch_size、gradient_accumulation_steps。

注: 10000条数据在当前para下完成一个epoch需要步数; $10000/48/4=52$ step/epoch;
 $10000/32/4=78$ step/epoch

5. 中文语言模型的评测基准

1. LLM 实时排行, 来自 UC伯克利: <https://lmsys.org/blog/2023-06-22-leaderboard/>
2. 选择中文模型: 中文语言理解测评基准(CLUE): <https://www.cluebenchmarks.com/index.html> 和 SuperCLUE琅琊榜: <https://www.superclueai.com/>

开源领域 ChatGLM, LLAMA, RWKV 主要就是这3种模型，中文好一点就是 ChatGLM，潜力最好的就是 LLAMA，RNN架构决定RWKV有很好的推理效率（随输入长度内存占比线性自增，而LLAMA则是指数增加）和 Length Extrapolation（关于长度外推性，可以参考苏神的文章）。当然 MPT-7B-StoryWriter-65k+ 模型也有较长的外推能力。

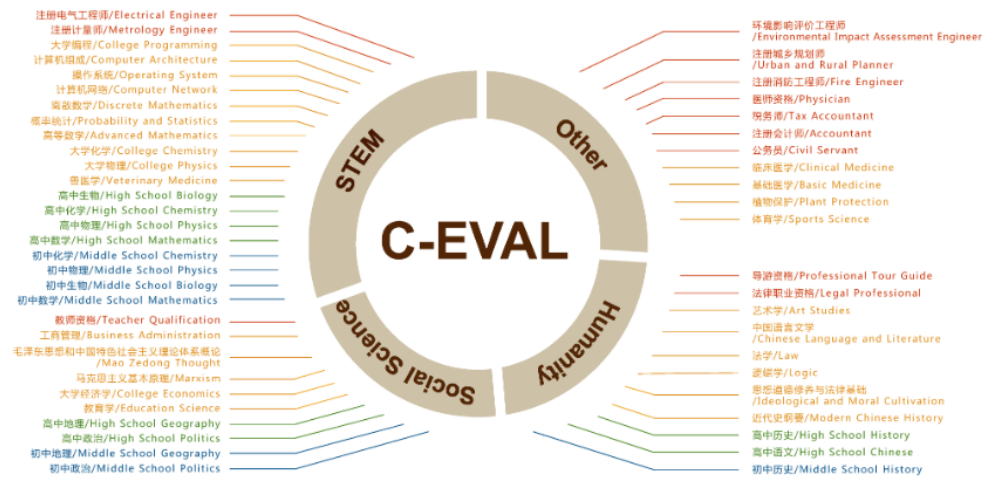
自ChatGPT为代表的大语言模型（Large Language Model, LLM）出现以后，由于其惊人的类通用人工智能（AGI）的能力，掀起了新一轮自然语言处理领域的研究和应用的浪潮。尤其是以ChatGLM、LLaMa等平民玩家都能跑起来的较小规模的LLM开源之后，业界涌现了非常多基于LLM的二次微调或应用的案例。下面这个项目在收集和梳理中文LLM相关的开源模型、应用、数据集及教程等资料，目前收录的资源已达100+个！

Awesome Chinese LLM，主要是整理开源的中文大语言模型，以规模较小、可私有化部署、训练成本较低的模型为主，包括底座模型，垂直领域微调及应用，数据集与教程等。

Awesome Chinese LLM 的GitHub地址：<https://github.com/HqWu-HITCS/Awesome-Chinese-LLM>

3. C-Eval:构造中文大模型的知识评估基准，其榜单是一个全面的中文基础 模型评估 套件(多层次、多学科的语文评价基础模型套件)。它由13948个选择题组成 问题跨越52个不同的学科和四个难度级别，测试集用于模型评估（简单来说就是针对中文模型的综合测试机），目的是C-Eval能够帮助开发人员跟踪模型开发的进展，以及分析开发中模型的优点和弱点。

C-Eval 的GitHub地址：<https://github.com/hkust-nlp/ceval>，论文地址：<https://arxiv.org/pdf/2305.08322v1.pdf>



不同颜色的主体表示四个难度等级：初中、高中、大学和专业。

比较有代表性，很多新出的模型或者微调过的模型都会在这样的一个基准上进行评估。榜单地址：<https://cevalbenchmark.com/static/leaderboard.html>

#	Model	Creator	Access	Submission Date	Avg	Avg(Hard)	STEM	Social Science	Humanities	Others
0	Qwen-72B	Alibaba Cloud	Weight	2023/11/30	82.8	64.7	77.1	91.7	84.7	82.9
1	Yi-34B	零一万物	Weight	2023/11/2	81.4	58.7	73.7	89.6	84.6	84.9
2	OrionStar-Yi-34B-Chat	OrionStarAI	Weight	2023/11/22	78.1	55.8	70.1	88	80.7	80.9
3	YAYI2-30B	中科院歌	Weight	2023/12/18	75.3	53.1	67.2	83.8	80.6	76.8
4	xDAN-L2-Chat-llm-v1.0	xDAN-AI	API, Private	2023/12/17	74.3	50.7	66.5	84.8	78.1	75.3
5	BlueLM-7B	vivo	Weight	2023/11/7	73.3	48.9	64.3	83.3	76.5	77.1
6	XVERSE-65B-2	XVERSE Technology	Weight	2023/12/8	72.4	50.8	65.7	85	74	71.8
7	Qwen-14B	Alibaba Cloud	Weight	2023/9/22	72.1	53.7	65.7	85.4	75.3	68.4
8	Yi-6B	零一万物	Weight	2023/11/2	72	46.6	62.3	83.9	76.3	74.6
9	XuanYuan-70B	度小满AI-Lab	Weight	2023/9/21	71.9	53.6	67.7	83.3	73.9	67.4
10	ChatGLM3-6B-base	Tsinghua & Zhipu AI	Weight	2023/10/26	69	46.8	61	82.4	73.4	66.9

其使用的数据集的地址是：<https://cevalbenchmark.com/static/leaderboard.html>，都是一些选择题。

Subset

Split

accountant (497 rows)

test (443 rows)

Search this dataset

id	question	A	B	C	D	answer
int32	string · lengths	string · lengths	string · lengths	string · lengths	string · lengths	string · clas
						
0 442	14 280	1 110	1 104	1 114	1 94	1 value
0	下列关于资本结构理论的说法中，不正确的是...	代理理论、权衡理论、有企业所得税条件下的...	按照优序融资理论的观点，考虑信息不对称和...	权衡理论是对有企业所得税条件下的MM理论的...	代理理论是对权衡理论的扩展	
1	某烟草公司2022年1月8日向烟农支付烟叶收购...	11.6	12.76	13.6	14.96	
2	下列关于企业价值评估的现金流量折现模型的...	现金流量折现模型是企业价值评估使用最广...	实体现金流量只能用企业的加权平均资本成本...	由于股利比较直观，因此股利现金流量模型在...	企业价值等于预期价值加上后续期价值的现...	